电 子 科 技 大 学

UNIVERSITY OF ELECTRONIC SCIENCE AND TECHNOLOGY OF CHINA

硕士学位论文

MASTER THESIS

论文题目 面向大规模低轨卫星的在网计算研究

学科专业	信息与通信工程
学　　号	**202321011337**
作者姓名	龚科成
指导教师	孙罡　　教　授
学　　院	信息与通信工程学院

摘　要

大规模低轨卫星组成的巨型星座即将成为新一代通信系统的基础设施，它为世界提供给了一种新的连接范式，一方面将全球各地的偏远地区的人相连，另一方面也为大规模全球范围内的物联网提供了新的机遇。

关键词：大规模低轨卫星，在网计算，机器学习，物联网

ABSTRACT

When Yang was a teenager, his mother contracted a disease and

Keywords: Martial arts, apostasy, revenge, fighting against the enemy, martial arts, apostasy, revenge, fighting against the enemy

目 录

第一章 绪论 1
1.1 研究背景与意义 1
1.1.1 大规模低轨卫星网络的发展浪潮与战略地位 1
1.1.2 LEO 卫星网络面临的严峻安全威胁：链路洪泛攻击 2
1.1.3 传统安全防御体系在 LEO 环境中的局限性 3
1.1.4 可编程数据面与在网计算赋能星上内生安全 3
1.2 国内外研究现状 4
1.2.1 卫星网络安全与动态路由技术研究现状 4
1.2.2 链路洪泛攻击检测与缓解技术研究现状 5
1.2.3 基于数据面可编程的在网安全防御研究现状 6
1.2.4 现状评述与本文的研究切入点 7
1.3 本文主要研究内容 7
1.3.1 面向大规模低轨卫星网络的多签名在网检测与单点缓解方法 8
1.3.2 基于抑制型扩散的多星协同在网推回缓解机制 8
1.4 本文的结构安排 8
第二章 相关概念与技术基础 10
2.1 大规模低轨卫星网络体系架构与特性 10
2.1.1 巨型星座的网络拓扑与星间链路模型 10
2.1.2 卫星网络的高动态性与快照路由模型 11
2.1.3 面向星上处理的软硬件资源约束 11
2.2 链路洪泛攻击的攻击模型与对抗分析 12
2.2.1 DDoS 攻击的演进与 LFA 的定位 12
2.2.2 Crossfire 攻击的构造模型 13
2.2.3 LEO 场景下 LFA 的退化策略 14
2.3 在网计算与 P4 可编程数据面架构 14
2.3.1 从 OpenFlow 到 P4：数据面可编程能力的演进 14
2.3.2 PISA 架构与匹配-动作流水线 15
2.3.3 数据面有状态处理与存储约束 16
2.3.4 面向在网安全的近似数据结构 17

2.4 分布式 Gossip 协议与协同防御基础 18
2.4.1 去中心化协同的必要性与传统共识的代价 18
2.4.2 Gossip 协议的基本原理与传播特性 18
2.4.3 抑制型传播与开销控制 19
2.5 本章小结 20
第三章 面向大规模低轨卫星网络的多签名在网检测与队列缓解方法 21
3.1 引言 21
3.2 问题定义与方法设计 22
3.2.1 威胁模型与问题表述 22
3.2.2 可分性分析与退化动机 23
3.2.3 系统总览与数据流解释 25
3.2.4 多签名可疑度建模与判决 26
3.2.5 FO/FI 在网近似统计结构 27
3.2.6 多签名在网检测与队列调度流程 28
3.3 实验设置与结果分析 29
3.3.1 实验平台与基线方案 29
3.3.2 数据集、流量生成与关键参数 29
3.3.3 评价指标与绘制方式 30
3.3.4 结果分析：退化证据链与多签名收益 31
3.3.5 消融实验与参数敏感性分析 33
3.4 本章小结 34
第四章 面向大规模低轨卫星网络的多星协同在网计算与推回缓解方法研究 37
4.1 引言 37
4.2 基于抑制型邻居扩散的多星协同在网计算方法 38
4.2.1 问题模型与设计目标 38
4.2.2 系统总体架构：CoopSatShield 39
4.2.3 本地证据抽取：从“单星检测”到“可协同的证据单元” 40
4.2.4 抑制型邻居扩散协同：避免信令风暴的 gossip 设计 41
4.2.5 协同判定与推回缓解：从“受害链路”推回到“上游入口” 44
4.2.6 信令开销与可实现性分析 46
4.3 仿真与分析 48
4.3.1 实验设置 48
4.3.2 结果与分析 50

4.4 本章小结 ……54
参考文献 ……57
附录 A 建模证明 ……60

第一章 绪论

1.1 研究背景与意义

1.1.1 大规模低轨卫星网络的发展浪潮与战略地位

传统地面通信网络虽然在人口密集区域实现了高度发达的覆盖，但全球仍有约 75% 的陆地面积和逾 90% 的海洋区域处于蜂窝网络无法覆盖的”数字盲区”[1]。在卫星通信领域，地球静止轨道（Geostationary Earth Orbit, GEO）卫星部署于约 36 000 km 的赤道正上方，虽然单星可覆盖约三分之一的地球可视面，但其固有的高轨道高度导致星地信号往返传播时延（Round-Trip Time, RTT）不低于约 $2 \times 36000/c \approx 240\,\text{ms}$（$c$ 为光速），这对实时视频通信、高频金融交易和交互式在线服务等延迟敏感型应用而言是不可接受的。中地球轨道（Medium Earth Orbit, MEO）卫星虽在时延上有所改善，但单星覆盖面积的缩减使其组网成本急剧攀升，至今仅有导航类（GPS/Galileo）和少数通信星座采用该轨道。

低地球轨道（Low Earth Orbit, LEO）的轨道高度通常介于 $h = 500\,\text{km}$ 至 2 000 km 之间。在该高度上，单跳星地传播时延可降至 $\tau_{\text{prop}} = h/c \approx 1.7\,\text{ms}\sim 6.7\,\text{ms}$，往返时延低于 15 ms，首次使卫星通信在延迟指标上具备了与地面光纤网络竞争的潜力。得益于运载火箭可重复使用技术（如 SpaceX Falcon 9 的一级回收）与卫星批量化制造模式的成熟，单颗 LEO 卫星的发射边际成本已从传统 GEO 卫星时代的数亿美元级降至百万美元以下 [1]。这一经济性突破催生了以 Starlink（截至 2025 年已部署逾 7 000 颗）、Amazon Kuiper（计划 3 236 颗）、OneWeb（648 颗）及中国 GW 星座（约 13 000 颗）为代表的”巨型星座”（Mega-constellation）建设浪潮 [1, 2]。

现代巨型星座的一个本质演进在于星间链路（Inter-Satellite Link, ISL）的引入。Starlink 自 v1.5 批次起为每颗卫星安装四部激光 ISL 终端，单链路容量可达 100 Gbps 量级 [3]。通过 ISL，星座中的卫星可直接进行多跳数据中继与路由，无需将每一跳数据下行至地面信关站再上行。这使得 LEO 星座从传统的”空中基站阵列”演变为一张具有独立路由与转发能力的”太空骨干网”（Internet in Space）。在某些洲际通信场景中（例如伦敦至纽约），由于光在真空中的传播速度约为光纤中的 1.47 倍（$c_{\text{vacuum}}/c_{\text{fiber}} \approx 3 \times 10^8/2.04 \times 10^8$），经由 LEO ISL 中继的路径甚至可以获得比海底光缆更低的端到端时延 [4]。正因如此，大规模 LEO 卫星网络已经上升为下一代天地一体化信息网络与第六代移动通信（6G）愿景中不可或缺的战略基础设施 [1]。

1.1.2 LEO 卫星网络面临的严峻安全威胁：链路洪泛攻击

作为一个向全球无差别提供接入服务的开放式基础设施，大规模 LEO 卫星网络不可避免地继承了互联网体系结构固有的安全脆弱性，并因其独特的物理特征而面临比地面网络更为严峻的威胁。具体而言，LEO 星座的物理结构是高度公开和可预测的：卫星的轨道根数（Two-Line Element, TLE）由 NORAD 和 CelesTrak 等机构向全球公开发布，其运动轨迹在未来数年内可被精确推算。这种可预测性虽然便利了网络的频率协调与链路预测，但同时也为恶意攻击者提供了精确的" 靶场信息" [5]。

在众多的网络层安全威胁中，链路洪泛攻击（Link Flooding Attack, LFA）对 LEO 星座网络构成了最为严峻的威胁之一。LFA 的典型代表为 Crossfire 攻击 [6]，其本质是一种高度隐蔽的间接拒绝服务（Indirect Denial of Service）攻击变种。与传统 DDoS 攻击直接向受害服务器倾泻海量流量不同，LFA 的攻击目标并非任何终端设备，而是网络内部承载大量转发业务的关键中间链路（Target Link） [6, 7]。

LFA 的实施机制如下：攻击者首先通过分布式路径探测（如从受控 Bots 向公共服务器发起 Traceroute）绘制目标网络的内部路由拓扑。随后，攻击者从中甄选出一组承载流量最为集中的瓶颈链路集合 $E_{\text{target}} \subset E$ [6]。在攻击执行阶段，攻击者操纵大量受控僵尸主机（Bots）集合 $\mathcal{B} = \{B_1, B_2, \ldots, B_N\}$，向分布在全球各地的合法公共服务器（Decoys）集合 $\mathcal{D} = \{D_1, D_2, \ldots, D_M\}$ 发起看似正常的低速（Low-rate）连接。由于路径选择的收敛效应，大量 $B_i \to D_j$ 通信对的路由路径会在 E_{target} 上重叠汇聚。当汇聚流量的总速率超过瓶颈链路容量 C_e 时，即 $\sum_{(i,j):e\in\text{path}(B_i,D_j)} r_{ij} > C_e$，链路 $e \in E_{\text{target}}$ 即发生拥塞，导致途经该链路的所有合法业务遭受严重的丢包与延迟退化。

LFA 对 LEO 星座网络的危害尤为严重，原因在于以下两点：第一，星间链路的带宽资源相对有限（通常为 10∼100 Gbps 量级），远低于地面骨干网的 Tbps 级容量，使得拥塞阈值更容易被突破。第二，由于卫星轨道参数完全公开，攻击者可以利用星座仿真工具（如 ICARUS [5] 等 LEO 攻击模拟框架）在攻击前精确计算出任意未来时刻的瓶颈链路位置，并预先规划 Bot-Decoy 通信对的路径选择。更为棘手的是，LEO 拓扑的周期性时变特性使得攻击者可以实施" 滚动式"（Rolling）攻击策略——随着星座拓扑的演进，攻击者不断切换攻击目标链路，使防御方难以建立持续有效的防护。近期研究进一步揭示，攻击者可利用 LEO 拓扑的时变特性，在路径时延切换过程中将分散发送的攻击流量在目标链路上实现时间聚焦，从而以更低的攻击成本达到相同的拥塞效果 [5, 8]。

1.1.3 传统安全防御体系在 LEO 环境中的局限性

面对 LFA 对 LEO 网络构成的生存级威胁，直接将地面网络中成熟的安全防御体系迁移至太空环境将遭遇两个维度的根本性矛盾。

矛盾一：控制环路时延与攻击自适应速度之间的不对称。传统的网络异常检测与缓解高度依赖软件定义网络（Software-Defined Networking, SDN）的集中式架构。其典型工作流程为：数据面交换机周期性地将流量统计信息（如 sFlow/NetFlow 采样数据）上报至远端控制器，控制器运行检测算法并计算缓解策略后，再将流表规则（Flow Rules）下发至数据面执行。在地面数据中心网络中，控制器与交换机之间的通信时延通常在 $\tau_{\text{ctrl}} < 1\,\text{ms}$ 量级，这一闭环可以在攻击演化之前完成响应。然而，在 LEO 卫星网络中，星地控制环路的 RTT 受限于物理传播距离，至少为 $2h/c \approx 3.3\,\text{ms}$（$h = 500\,\text{km}$ 最低轨道），若涉及多跳 ISL 转发或地面网络路由，实际 RTT 通常达到 20∼200 ms。相比之下，LFA 攻击者可以在毫秒级时间尺度内通过 C&C（Command and Control）指令重组僵尸主机集合或微调各流的发送速率。这种时延不对称使得依赖地面控制器的” 感知—上传—决策—下发” 闭环在 LEO 环境中始终处于滞后状态，防御策略抵达卫星时攻击特征已发生漂移 [6, 9]。

矛盾二：星载硬件的物理极限与传统安全引擎的资源需求之间的鸿沟。 LEO 卫星的设计与发射受到尺寸、重量与功耗（Size, Weight, and Power, SWaP）的严苛约束。卫星的电力供应完全依赖有限面积的太阳能电池阵列（典型功率预算为数百瓦至千瓦级），热控系统仅能依靠被动辐射散热。因此，星载处理器通常采用高能效比的抗辐射 FPGA 或宇航级 ASIC，其计算能力和片上存储容量显著低于地面商用设备。以片上 SRAM 为例，地面高性能可编程交换芯片（如 Intel Tofino 2）可提供约 ∼80 MB 的片上 SRAM，而宇航级芯片的可用 SRAM 通常仅为数 MB 量级 [10]。传统的网络安全引擎（如基于深度学习的异常检测系统或维护 Per-flow 状态表的流量清洗设备），其运行所需的计算量和内存容量远远超出了星载平台的供给能力。在数十 Gbps 的链路吞吐率下，以 Per-flow 粒度维护所有活跃流（可达 $10^6 \sim 10^7$ 条）的精确状态，所需的内存空间约为数百 MB 至 GB 量级，这在星载环境中是不可实现的 [10, 11]。

1.1.4 可编程数据面与在网计算赋能星上内生安全

上述两重矛盾指向一个核心推论：有效的 LEO 防御方案必须满足两个基本约束——（1）在数据面就地完成检测与缓解的闭环，绕过高时延的星地控制环路；（2）算法的计算复杂度与状态空间消耗必须严格适配星载硬件的 SWaP 边界。近年来，可编程数据平面（Programmable Data Plane）与在网计算（In-Network Computing,

INC）技术的发展为满足上述约束提供了可行的技术基座 [12, 13]。

P4（Programming Protocol-independent Packet Processors）语言赋予了网络交换芯片在维持纳秒级线速（Line-rate）转发性能的同时，执行由用户自定义的包处理逻辑的能力。P4 程序运行于协议无关交换架构（Protocol Independent Switch Architecture, PISA）的硬件流水线之上。在该架构中，数据包经过可编程的解析器（Parser）提取报文头字段后，依次通过由多级匹配-动作单元（Match-Action Unit, MAU）组成的入口流水线（Ingress Pipeline）和出口流水线（Egress Pipeline）。每个 MAU 级可在单个时钟周期内完成一次 TCAM/SRAM 查表与一组 ALU 算术逻辑操作。通过在流水线中部署寄存器（Registers）、计数器（Counters）等有状态存储元素，交换芯片能够以 $O(1)$ 的时间复杂度对每个经过的数据包进行状态化处理 [14]。

在网计算的核心理念是：将特定的高频、轻量级计算任务从终端或控制器”卸载”（Offload）至网络内部的交换节点，使计算与转发在数据路径上融为一体 [12]。将这一理念应用于 LEO 卫星安全防御，可实现两方面的结构性优势。其一，星载 P4 交换机可在数据面就地完成流量特征提取、异常判定与丢包缓解的全闭环操作，形成”零额外时延”的本地响应能力，从根本上规避了星地控制环路的时延瓶颈。其二，基于流水线架构的近似数据结构（如 Bloom Filter、Count-Min Sketch）仅消耗数百 KB 至数 MB 的片上 SRAM，即可实现对大规模流量的高效统计与追踪，天然契合星载平台的 SWaP 约束 [10, 11]。

综上，利用在网计算技术在 LEO 卫星的数据面构建原生的、内生的安全检测与缓解能力，将对抗 LFA 攻击的第一道防线推进至网络内部的转发节点，是解决上述两重矛盾的根本性技术路径。这一判断构成了本论文立题的核心研究动机 [10]。

1.2 国内外研究现状

1.2.1 卫星网络安全与动态路由技术研究现状

LEO 卫星网络的路由与安全是两个交织发展但进度严重失衡的研究方向。在路由方面，由于卫星的高速运动（LEO 轨道速度约 7.5 km/s，轨道周期约 90～120 min）导致物理拓扑持续变化，传统的基于链路状态通告（Link State Advertisement, LSA）的路由协议（如 OSPF）若直接套用，将产生每分钟数百至数千次的路由重收敛事件，造成控制面信令风暴。为此，学术界提出了多种适应性方案：（1）基于离散时间快照（Snapshot）的路由预计算方案，利用轨道可预测性将连续时间划分为一系列离散时段，在每个时段开始时加载预计算的转发表（Forwarding Information Base, FIB），避免实时路由收敛 [4]；（2）基于虚拟拓扑

（Virtual Topology Routing）的方案，将物理 ISL 拓扑映射为逻辑上稳定的虚拟节点关系，在逻辑层面维护路由表；（3）基于地理位置的分布式寻址路由，利用卫星的地理坐标作为路由标识，实现无需全局拓扑信息的贪婪转发。

相比之下，LEO 网络的安全防御研究发展严重滞后。现有研究大量集中于卫星-地面链路的接入认证（如基于物理层特征的认证、抗量子密钥分发协议）和信道加密（如适应高时延的 TLS/DTLS 优化），而针对星间链路层面的数据面攻击防御——特别是 DDoS 和 LFA 类流量攻击——的在轨（In-orbit）研究几乎处于空白 [15]。目前工程实践中的主流做法仍是" 将可疑流量绕路至地面流量清洗中心"（Traffic Scrubbing via Ground Rerouting），即通过星地链路将攻击流量引导至地面的高性能清洗设施进行过滤后再回注。这一架构存在三个固有缺陷：（1）绕路增加了额外的星地传播时延（至少一个 RTT，约 3～15 ms），与 LFA 的毫秒级自适应能力不匹配；（2）星地下行链路本身的带宽有限，当遭受大规模攻击时，下行回传通道本身可能成为新的拥塞瓶颈；（3）这一方案对地面基础设施的依赖使其在星地链路中断（如极端天气或对抗性干扰）时完全失效 [15, 16]。

1.2.2 链路洪泛攻击检测与缓解技术研究现状

针对 LFA 这一高级威胁，地面互联网领域已发展出多个具有代表性的防御框架，可按其技术路线划分为三大类别。

第一类：基于主动测量的检测方案。以 LinkScope [17] 为代表，其核心思想是由网络运维方主动向可疑链路注入探测包（Probe Packets），通过分析探测包的丢包率、时延抖动等端到端性能指标来推断链路是否正在遭受洪泛攻击。该类方案的主要局限在于：（1）探测流量本身占用链路带宽，在带宽资源极为稀缺的 ISL 上会加剧拥塞；（2）在 LEO 的高动态拓扑下，探测路径会因拓扑切换而频繁失效，导致测量结果不连续；（3）LFA 攻击者可以通过控制攻击强度使其刚好低于探测阈值，实现规避。

第二类：基于集中式 SDN 控制器的检测与流量工程缓解方案。以 Woodpecker [18] 和 RADAR [19] 为代表。Woodpecker 通过 SDN 控制器集中采集全网交换机的流量统计信息，构建全局流量矩阵（Traffic Matrix, TM），利用矩阵分解与统计异常检测识别受攻击链路，随后通过求解线性规划（Linear Programming, LP）问题计算最优的流量重路由策略。RADAR 在此基础上引入了基于博弈论的自适应对抗模型，将攻防双方建模为零和博弈（Zero-sum Game），通过纳什均衡（Nash Equilibrium）求解最优防御策略。此类方案的核心瓶颈在于其对中心控制器的强依赖：（1）全局流量矩阵的收集周期通常为秒级，加上 LP 求解的计算耗时，端到端的响应时延远超 LFA 的自适应速度；（2）在 LEO 环境中，星地控制环路

的额外时延进一步恶化了响应速度；（3）控制器本身可能成为单点故障（Single Point of Failure）。

第三类：基于 P4 可编程数据面的分布式检测方案。Ripple [20] 提出了一种在 P4 交换机上实现的去中心化 LFA 防御框架，通过在每台边缘交换机的数据面部署轻量级的流量监控逻辑，避免了对集中式控制器的依赖，并利用 P4 的线速处理能力实现了毫秒级的响应速度。Mew [21] 进一步扩展了 P4 数据面的检测能力，引入了基于可编程交换机的大规模动态 LFA 防御机制，支持在交换机间协同共享检测状态。然而，这两个方案均是针对地面数据中心或 ISP 骨干网设计的，其隐含假设包括：（1）交换机具有充裕的 SRAM/TCAM 资源；（2）网络拓扑高度稳定，设备间的协同通信可靠且低延迟；（3）可以灵活地通过控制面动态更新 P4 程序和流表。这些假设在 LEO 环境中均不成立。特别是，当面对 LFA 的退化策略（如多靶机分摊、低速微流化）时，现有的 P4 方案仅依赖聚合速率（Rate-only）作为单一检测特征，无法有效区分隐蔽的 LFA 流与合法的长尾大流（Heavy Hitters），导致显著的漏报率和误报率 [20, 21]。

1.2.3 基于数据面可编程的在网安全防御研究现状

P4 可编程技术在网络安全领域的应用已取得显著进展，主要集中在以下方向。

在网络遥测方面，带内网络遥测（In-band Network Telemetry, INT）技术利用 P4 的协议无关性，在数据包头中嵌入沿途各交换机的内部状态信息（如入口/出口时间戳、队列占用深度、链路利用率等），实现了对网络状态的线速、逐包、全路径可视化。INT 为异常检测提供了前所未有的细粒度数据源，但其在 LEO 环境中的应用面临挑战：INT 信息的嵌入增加了数据包的开销（每跳通常增加 8～16 Bytes），在带宽受限的 ISL 上可能并不稳定 [14]。

在数据面异常检测方面，已有多项工作探索了在 P4 交换机上实现实时 DDoS 检测的可行性。例如，ddosd-p4 项目在 P4 数据面上实现了基于信息熵（Shannon Entropy）的异常检测：利用 Count-Sketch 数据结构对源/目的 IP 的频次分布进行近似估计，通过预计算的离散化查找表（LPM Table）将频次映射为熵贡献值，并在观测窗口结束时利用 EWMA（指数加权移动平均）与 EWMMD（指数加权移动平均绝对偏差）进行统计异常判定。该方案证明了在 P4 数据面实现统计检测的技术可行性，但其熵检测依赖于攻击流量导致 IP 分布发生显著偏移这一假设——在 LFA 的多靶机分散策略下，攻击流量的目的 IP 分布可能被刻意分散化，使得熵值变化不显著，从而导致漏检 [11, 22]。

在数据面缓解方面，基于 P4 的可编程防火墙和速率限制器已在地面网络中得到验证 [11, 23]。通过 P4 的 Meter 原语，交换机可对匹配特定规则的流量执行令牌

桶（Token Bucket）限速或直接丢弃，实现线速级的攻击流量过滤。然而，这些方案通常假设攻击流量可以通过五元组（5-tuple）或源 IP 前缀等简单规则精确匹配——这一假设在 LFA 场景中并不成立，因为每条攻击微流都使用合法的源地址和目的地址，且流五元组与正常业务无异 [6]。此外，Patronum [24] 和 ACC-Turbo [23] 分别从体量化检测和聚合拥塞控制的角度进一步验证了可编程交换机在 DDoS 防御中的实用性，但同样面临 LFA 微流隐蔽性带来的特征匹配难题。

1.2.4 现状评述与本文的研究切入点

综合上述三个方向的研究现状，可以提炼出当前 LEO 网络 LFA 防御面临的三重核心矛盾：

（1）**时延矛盾**：地面集中式防御方案的端到端响应时延（秒级）远慢于 LFA 攻击自适应切换速度（毫秒级），在 LEO 星地控制环路的额外时延下进一步恶化。

（2）**视野矛盾**：LFA 攻击流量以碎片化形式分散穿越多条 ISL，任何单颗卫星仅能观测到途经本节点的局部流量切片，无法仅凭局部信息准确区分攻击与正常业务。

（3）**资源矛盾**：有效检测隐蔽 LFA 需要提取多维度流量特征（如扇入度、扇出度、持续性等），而 P4 流水线的计算能力和 SRAM 容量严格受限于 SWaP 约束，传统的多特征检测方案面临状态爆炸风险。

上述三重矛盾在已有文献中已得到不同程度的揭示 [10, 20, 21]。面对上述三重矛盾，本文摒弃传统的" 感知—上传—中央清洗" 范式，确立了**轻量级数据面在网状态近似**与**不依赖中心控制的多星去中心化协同**相结合的技术路线作为核心研究切入点。其学术目标具体分解为两个关键问题：

- **问题一**（对应第三章）：如何在 P4 流水线的严苛计算与存储约束下，设计一种兼顾低资源开销与高检测精度的多维度流量特征在网提取方法，实现对隐蔽 LFA 攻击的单星实时检测与就地缓解？
- **问题二**（对应第四章）：如何在无中心控制器参与的条件下，设计一种低信令开销、高鲁棒性的多星分布式协同机制，将单星的局部检测结果聚合为全网一致的攻击判定，并实现从受害链路向攻击源头方向的逐跳推回缓解？

1.3 本文主要研究内容

立足上述研究动机与问题定义，本论文利用 LEO 卫星网络高度结构化且可预测的时变拓扑特性，以在网计算为核心技术手段，为大规模 LEO 星座构建了一套

从单节点检测到多节点协同推回的系统性安全防御体系。主要研究内容包括以下两个方面。

1.3.1 面向大规模低轨卫星网络的多签名在网检测与单点缓解方法

本部分研究（对应第三章）针对单节点在 P4 数据面上进行 LFA 检测时面临的特征退化困境，提出了一种多签名在网检测与队列缓解方法，称为 MS-SatShield（Multi-Signature Satellite Shield）。该方法的核心创新在于设计了一种级联式的两级检测流水线架构：第一级利用基于 Count-Min Sketch 的轻量级 Top-k 重流检测算法，以极低的状态开销（$O(w \times d)$，其中 w 为 Sketch 宽度、d 为哈希函数行数）从海量流量中快速锁定速率异常的候选流集合；第二级在候选集上部署双缓冲位图（Double-buffered Bitmap）结构，对每个候选流在线提取发散入度（Fan-in）、发散出度（Fan-out）和历史持续性（Persist）等多维结构特征签名。多签名的联合评分机制使得即便攻击者通过退化策略使单一维度的特征” 隐身”，仍可被其他维度的异常所捕获。在缓解侧，本方法设计了一种基于动态降权的自适应队列惩罚策略，根据多签名综合可疑度对可疑流进行差异化的排队优先级调整，在保护合法业务的同时对攻击流施加精准的吞吐抑制。

1.3.2 基于抑制型扩散的多星协同在网推回缓解机制

本部分研究（对应第四章）针对单星检测视野局限性与” 被动就地丢弃” 模式的不足，提出了一种多星协同在网计算与逐跳推回缓解方法，称为 CoopSatShield（Cooperative Satellite Shield）。该方法包含两个核心组件：（1）抑制型流言传播协议（Suppressed Gossip Protocol），通过在 LEO 星座的 Mesh 拓扑中设计一种受控的、基于事件触发且带冗余抑制的邻居间信息扩散机制，在严格限制信令开销的前提下实现局部区域内多颗卫星之间的攻击证据快速交叉验证与聚合；（2）逐跳推回缓解机制（Hop-by-hop Pushback），一旦区域内的多星协同判定达成共识，受害链路的卫星节点将生成携带攻击特征签名的推回令牌（Pushback Token, PT），沿攻击流的反向路径逐跳回传至上游节点，指示路径上的各卫星在其数据面对匹配的攻击流执行限速或丢弃操作。这一机制使防御阵线从被动的” 受害点丢弃” 转变为主动的” 向源头推进”，有效减少了攻击流在网络中的无效传输，提升了全网的带宽利用效率。

1.4 本文的结构安排

本论文共分为六章，各章内容安排如下：

第一章绪论。阐述了大规模 LEO 卫星网络的发展现状及其面临的 LFA 安全威

胁，分析了传统防御体系在 LEO 环境中的根本性局限，引出了基于在网计算的星上内生安全防御思路。在此基础上，系统综述了卫星网络安全、LFA 防御和 P4 在网安全三个方向的国内外研究现状，梳理了现有工作的不足与研究空白，明确了本文的研究切入点和主要研究内容。

第二章相关概念与技术基础。为后续研究奠定理论基础。详细介绍了 LEO 巨型星座的网络拓扑模型与快照路由机制，深入分析了 LFA 的攻击构造模型与退化策略，系统阐述了 P4/PISA 可编程数据面架构的工作原理、计算约束与有状态处理能力，并引入了分布式 Gossip 协议及其抑制型变种的理论基础。

第三章面向大规模低轨卫星网络的多签名在网检测与队列缓解方法。阐述 MS-SatShield 的系统设计、算法细节与实验验证。重点介绍基于 Top-k 候选锁定与 FI/FO/Persist 多签名在网提取的级联检测流水线，以及基于动态降权的队列缓解策略，并通过大规模仿真实验验证其在多种 LFA 退化策略下的检测精度与吞吐恢复效果。

第四章面向大规模低轨卫星网络的多星协同在网计算与推回缓解方法研究。阐述 CoopSatShield 的协同框架设计与实验评估。重点介绍抑制型 Gossip 协议的信令开销控制机制、多星证据交叉验证的判定逻辑，以及逐跳推回令牌的生成与传播机制，并通过跨区域仿真实验验证其在不同攻击规模和网络状态下的协同效率与可扩展性。

第五章总结与展望。总结全文的研究成果与主要贡献，讨论当前工作的局限性，并对 LEO 网络在网安全防御的未来研究方向进行展望。

第二章 相关概念与技术基础

本论文所提出的多签名在网检测（第三章）与多星协同推回缓解（第四章）方法，其设计与实现深度依赖于四个领域的理论支撑：大规模 LEO 星座的网络拓扑与约束模型、链路洪泛攻击的构造机理与对抗博弈、P4 可编程数据面的架构原理与计算边界、以及分布式 Gossip 协议的传播理论与开销控制。本章将逐一展开论述，为后续章节建立可直接引用的理论基础。

2.1 大规模低轨卫星网络体系架构与特性

2.1.1 巨型星座的网络拓扑与星间链路模型

现代 LEO 巨型星座的网络拓扑可从两个层面进行形式化描述：物理构型层面与网络图论层面。

在物理构型层面，巨型星座的卫星分布遵循 Walker 星座模型。以 Walker Delta 构型为例（如 Starlink 第一阶段主体），其参数化表示为 $W(\delta : T/P/F)$，其中 δ 为轨道倾角，T 为卫星总数，P 为轨道平面数，F 为相邻轨道间的相位因子。对于 Starlink Shell 1（$\delta = 53, T = 1584, P = 72, F = 1$），每个轨道平面承载 $T/P = 22$ 颗卫星，轨道高度 $h = 550\,\text{km}$。Walker Star 构型（如 Telesat Lightspeed，$\delta = 99.5$，近极地轨道）则在极地地区存在轨道交叉区，形成拓扑上的特殊汇聚点。两种构型的共同特征是：在同一轨道平面内，相邻卫星之间通过同轨星间链路（Intra-plane ISL）相连，链路长度相对稳定；不同轨道平面之间，通过异轨星间链路（Inter-plane ISL）相连，其长度随卫星纬度位置的变化而周期性变化 [3]。在高纬度区域（特别是极圈附近），异轨 ISL 由于相邻轨道平面间距过大或卫星相对速度过高，可能周期性中断 [1, 25]。

在网络图论层面，LEO 星座在任意给定时刻 t 的网络拓扑可建模为时变有向图 $G(t) = (V(t), E(t))$，其中 $V(t)$ 为当前在轨运行的卫星节点集合，$E(t)$ 为当前活跃的 ISL 集合。对于典型的四端口 ISL 设计（前后两条同轨 ISL、左右两条异轨 ISL），除极地交叉区外，每个卫星节点的网络度数约为 4。这种规则的格状（Grid/Torus）拓扑结构意味着：（1）任意两颗卫星之间的最短路径跳数可由其在网格中的曼哈顿距离（Manhattan Distance）近似估计；（2）当某条核心 ISL 发生拥塞时，替代路径的跳数增量相对可控，但同时也意味着拥塞可能沿网格的行或列方向传播，引发连锁效应 [4, 5]。

每条 ISL $e \in E(t)$ 具有有限的链路容量 C_e（通常为 10～100 Gbps 量级）。定义

链路 e 在时刻 t 的利用率为 $\rho_e(t) = \lambda_e(t)/C_e$，其中 $\lambda_e(t)$ 为 e 上的瞬时聚合流量速率。当 $\rho_e(t) > 1$ 时，链路进入拥塞状态，数据包在输出缓冲队列中排队等待或被丢弃。LFA 攻击的目标正是通过人为制造 $\rho_{e^*}(t) > 1$（$e^* \in E_{\text{target}}$）来使目标链路 e^* 瘫痪 [6]。这一量化模型为第三章的检测阈值设定和第四章的推回触发条件提供了形式化基础。

2.1.2 卫星网络的高动态性与快照路由模型

LEO 卫星以约 7.5 km/s 的速度运行（对于 $h = 550\,\text{km}$ 的圆轨道，轨道周期约 $T_{\text{orbit}} = 2\pi\sqrt{(R_E + h)^3/\mu} \approx 95.6\,\text{min}$，其中 $R_E = 6371\,\text{km}$ 为地球半径，$\mu = 3.986 \times 10^{14}\,\text{m}^3/\text{s}^2$ 为地心引力常数）。这一高速运动带来两个层面的动态性 [1]。

第一层面为星地覆盖的动态性：每颗卫星的地面覆盖区（Footprint）持续移动，地面终端需要每隔数分钟执行一次卫星切换（Satellite Handover）。第二层面为星间拓扑的动态性：异轨 ISL 的物理可用性和链路质量随卫星间相对位置的变化而波动；在极地区域，异轨 ISL 的周期性中断导致网络在该区域的连通性呈现间歇性特征。

为应对上述动态性，学术界广泛采用离散时间快照（Discrete-Time Snapshot）模型。其核心思想是利用 LEO 轨道运动的完全可预测性（由开普勒轨道力学确定），将连续时间轴划分为一系列等长的离散时段 $\{[\tau_k, \tau_{k+1}) | k = 0, 1, 2, \ldots\}$，时段长度 $\Delta\tau = \tau_{k+1} - \tau_k$（典型值为 10∼30 s，如 Starlink 约 15 s）。在每个时段 $[\tau_k, \tau_{k+1})$ 内，网络拓扑 $G(\tau_k)$ 被视为静态不变，卫星在时段开始时加载由地面网络管理系统预先计算并上注的转发信息表（Forwarding Information Base, FIB）。时段结束时，所有路由状态同步更新至下一时段的 FIB [4, 26]。

快照模型对本文的防御方案设计具有两方面的直接意义。其一，在第三章中，在网检测算法的状态累积周期（检测窗口 W）可以与快照时段 $\Delta\tau$ 对齐——在每次拓扑切换时，对检测状态表进行优雅的清洗（Graceful Reset），既避免了跨拓扑期间的状态污染，又保证了窗口内统计的一致性。其二，在第四章中，由于同一快照时段内的路由路径是确定的，推回令牌（Pushback Token）可以沿着已知的反向路径精确回传，无需实时的路径发现机制。

2.1.3 面向星上处理的软硬件资源约束

将安全检测与缓解逻辑部署于卫星平台，必须正视航天工程的 SWaP（Size, Weight, and Power）约束对计算与存储资源施加的刚性上限 [1, 10]。

在功耗维度，LEO 卫星的电力供应完全依赖太阳能电池阵列。以典型的中型通信卫星为例，其太阳能帆板的发电功率约为 1∼5 kW，而这一功率预算需在通信

载荷（星载天线、射频前端）、计算单元、姿态控制系统（ADCS）和热控系统之间分配。分配给网络交换与处理单元的功耗预算通常不超过数十瓦。作为对比，地面数据中心的高性能可编程交换芯片（如 Intel Tofino 2, 12.8 Tbps）的典型功耗约为 200∼300 W，仅此一项即可能超出整颗小型卫星的总功率预算 [1, 14]。

在存储维度，地面 Tofino 2 芯片提供约 80 MB 的片上 SRAM 和 40 Mb 的 TCAM。而宇航级 FPGA（如 Xilinx Virtex-5QV 系列抗辐射器件）的片上 BRAM 容量通常仅为数 MB 量级。如果以 Per-flow 粒度维护流状态表（假设每条流需要 ∼64 Bytes 的状态信息，活跃流数量为 N_f），则精确状态维护所需的内存为 $M_{\text{exact}} = 64N_f$ Bytes。当 $N_f = 10^6$（中等规模流量）时，$M_{\text{exact}} = 64$ MB，这已远超宇航级芯片的片上 SRAM 容量 [10, 14]。

在计算维度，PISA 流水线的每个 MAU 级仅能在单个时钟周期内执行有限的 ALU 操作：哈希计算、整数加减法、位移、按位逻辑运算（AND/OR/XOR）。不支持浮点运算、乘除法、条件循环（`for/while`）或递归调用。这意味着任何需要迭代收敛（如梯度下降）或复杂数学运算（如对数、三角函数）的算法均无法在数据面直接实现 [27]。

上述约束共同构成了本文算法设计的” 硬边界”：所有在网检测逻辑必须是单遍（Single-pass）的、无循环的，且状态空间消耗必须严格控制在 MB 量级以内。这一约束直接驱动了第三章中近似数据结构（Count-Min Sketch、Bitmap）与级联两级检测架构的设计选择 [10]。

2.2 链路洪泛攻击的攻击模型与对抗分析

2.2.1 DDoS 攻击的演进与 LFA 的定位

分布式拒绝服务（Distributed Denial of Service, DDoS）攻击的演进可划分为三个世代。第一代为体积型攻击（Volumetric Attacks），以 UDP 反射放大（如 DNS/NTP/Memcached 反射）和 SYN Flood 为代表，其目标是以绝对的流量体量耗尽受害端的接入带宽或处理能力。此类攻击特征明显（流量突增、协议异常），可被基于简单阈值和五元组黑名单的流量清洗中心有效拦截。第二代为应用层攻击（Application Layer Attacks），如 HTTP GET/POST Flood 和 Slowloris，其流量体量较小但精准打击应用层资源（如 Web 服务器的并发连接数或数据库查询能力）。此类攻击需要更细粒度的应用层检测逻辑，但攻击流量仍然汇聚于受害终端，终端侧的异常可被直接感知 [15]。

LFA 代表了第三代攻击范式的质变。其根本性的差异在于：攻击的受害者不再是任何终端节点，而是网络内部的中间链路（Target Link）。由于攻击流量的源

地址和目的地址均为合法地址，且每条流的速率和行为模式与正常业务无异，终端侧（无论是 Bot 还是 Decoy）均不会感知到异常。唯一的" 受害现场" 是拥塞的中间链路，而这恰恰是传统终端侧安全设备的监控盲区。这一特点使 LFA 成为当前已知隐蔽性最高的网络层攻击手段之一 [6, 7]。

2.2.2 Crossfire 攻击的构造模型

Crossfire [6] 是 LFA 的经典实例，其攻击过程可形式化为三阶段模型。

阶段一：拓扑测绘与瓶颈链路识别。攻击者控制分布在全球各地的僵尸主机集合 $\mathcal{B} = \{B_1, \ldots, B_N\}$，从每个 B_i 向目标区域附近的公共服务器集合 $\mathcal{D} = \{D_1, \ldots, D_M\}$（如 CDN 节点、DNS 根服务器）执行 Traceroute，从而获取 $N \times M$ 条路径信息。通过对这些路径的重叠分析，攻击者可构建出一张目标网络的链路使用频率图（Link Usage Frequency Map），并从中识别出边介数中心性（Edge Betweenness Centrality）最高的链路子集 E_{target}。对于任意链路 e，其边介数中心性定义为：

$$\mathrm{BC}(e) = \sum_{(i,j)} \frac{\sigma_{ij}(e)}{\sigma_{ij}} \tag{2-1}$$

其中 σ_{ij} 为 B_i 到 D_j 的最短路径总数，$\sigma_{ij}(e)$ 为其中经过链路 e 的路径数。$\mathrm{BC}(e)$ 值越高，表明该链路承载的路径汇聚度越大，则为 LFA 目标 [6]。

阶段二：Bot-Decoy 通信对分配。攻击者需要为每个 Bot B_i 分配一组目标 Decoy $\{D_{j_1}, D_{j_2}, \ldots\}$，使得所有 Bot-Decoy 通信对在目标链路 $e^* \in E_{\text{target}}$ 上的汇聚流量总和超过其容量。形式化地，攻击者需要求解以下可行性问题：

$$\sum_{\substack{(i,j):\\ e^* \in \mathrm{path}(B_i, D_j)}} r_{ij} \geq C_{e^*} \tag{2-2}$$

其中 r_{ij} 为 B_i 向 D_j 发送的流量速率。为了保持隐蔽性，每条流的速率需满足 $r_{ij} \leq r_{\text{thresh}}$（典型值为数十 KB/s 至数百 KB/s），使其不触发终端侧或中间节点的单流速率告警 [6]。

阶段三：协同流量投送。攻击者通过 C&C（Command and Control）信道同步启动所有 Bot 的流量发送。在 LEO 场景中，由于星座拓扑的快照切换周期 $\Delta\tau$ 通常为 10∼30 s，攻击者可将攻击持续时间与快照周期对齐，在每个快照时段内保持稳定的攻击流配置，随拓扑切换同步调整 Bot-Decoy 分配方案 [5, 6]。

2.2.3 LEO 场景下 LFA 的退化策略

在 LEO 环境中，攻击者可利用星座拓扑的公开性和规律性设计针对性的退化策略（Degrading Strategies），以规避各类防御机制。本文重点关注以下三种退化策略，它们直接驱动了第三章多签名检测方案的特征维度选择。

退化策略一：低速微流化（Low-rate Dilution）。攻击者通过扩大僵尸主机规模 N，将总攻击带宽分摊至更多的 Bot，使每条流的速率 r_{ij} 降至极低值。设目标链路容量为 C_{e^*}，正常背景流量为 λ_{bg}，则攻击成功所需的 Bot 总流量为 $\Lambda_{\text{atk}} = C_{e^*} - \lambda_{\text{bg}}$。若 N 足够大，则 $r_{ij} = \Lambda_{\text{atk}}/(N \cdot \bar{M}) \to 0$（$\bar{M}$ 为每个 Bot 的平均 Decoy 数），使得基于聚合速率的 Heavy-Hitter 检测完全失效 [6, 20]。

退化策略二：多靶机分散（Fan-out/Fan-in Relaxation）。攻击者增大每个 Bot 的目标 Decoy 数量和 Decoy 的地理分散度（即放大 Fan-out），同时增大指向同一 Decoy 的 Bot 数量（即放大 Fan-in），使攻击流量在源端和目的端均呈高度分散分布。这种分散性破坏了基于目的 IP 熵变化或目的前缀聚合的检测假设 [6]。

退化策略三：脉冲式滚动攻击（On-Off Rolling）。攻击者将攻击分为交替的”开”（On）和”关”（Off）阶段，在”开”阶段以全力发送攻击流量，在”关”阶段暂停发送，使链路利用率回落至正常水平。通过控制 On-Off 的周期与占空比，攻击者可以使基于时间窗口平均值的检测算法难以捕捉瞬时突增。此外，攻击者可以在不同的快照时段内切换目标链路，形成时间和空间上的双重游击 [5, 8]。

上述三种退化策略的共同效果是：使 LFA 攻击流量在单一特征维度上与正常业务流量难以区分。这一分析直接驱动了第三章中引入多维度特征签名（速率、Fan-in/Fan-out、持续性）的设计决策——任何单一维度均可被退化策略规避，但多维度的联合异常仍然可以被检测到 [10]。

2.3 在网计算与 P4 可编程数据面架构

2.3.1 从 OpenFlow 到 P4：数据面可编程能力的演进

软件定义网络（SDN）通过解耦控制平面与数据平面，首次实现了对网络转发行为的集中式可编程控制。其代表性南向接口协议 OpenFlow 定义了一组固定的匹配字段集合（如 12-tuple 和后续扩展的 OXM 匹配字段），控制器通过下发流表规则指示数据面对匹配的数据包执行预定义的动作（转发、丢弃、修改字段等）。OpenFlow 的根本局限在于其”协议依赖性”：可匹配的协议头字段由标准预先定义并硬编码于交换芯片中，无法支持自定义协议头的解析与处理。当需要处理新的协议（如 INT 遥测头）或自定义封装格式时，必须等待标准更新和芯片厂商的硬

件支持，这严重制约了数据面的创新速度 [28]。

P4（Programming Protocol-independent Packet Processors）语言 [29] 的出现解决了这一根本性问题。P4 的设计理念基于两个核心原则：协议无关性（Protocol Independence）——程序员可以自行定义数据包的解析逻辑和匹配字段，交换芯片不再绑定于任何特定协议；以及目标无关性（Target Independence）——同一份 P4 程序可以通过不同的编译器后端编译为不同硬件平台（如 ASIC 交换芯片、FPGA SmartNIC、软件交换机 BMv2）的可执行代码。P4 程序定义了完整的数据包处理流程：从比特流中解析出报文头（Parser），经过多级匹配-动作处理（Match-Action Pipeline），最终将修改后的报文头重新组装为比特流（Deparser）。

这种" 自顶向下" 的编程范式使得网络运营者和研究者能够根据需求灵活定义数据面的处理逻辑，无需受限于芯片硬编码的功能集。对于本文的研究而言，P4 的协议无关性意味着：我们可以在 LEO 卫星的交换芯片上自定义用于多签名检测的协议头部字段和状态更新逻辑，而无需依赖芯片厂商的专门支持 [14, 29]。

2.3.2 PISA 架构与匹配-动作流水线

P4 程序运行于协议无关交换架构（Protocol Independent Switch Architecture, PISA）之上 [27]。PISA 的硬件流水线由以下四个功能模块依次级联构成：

（1）可编程解析器（Programmable Parser）。解析器是一个有限状态机（Finite State Machine, FSM），负责将输入的原始比特流按照 P4 程序定义的协议格式依次提取（Extract）各层报文头字段，并将提取结果写入包元数据（Packet Metadata）和包头向量（Parsed Header Vector, PHV）中。解析器支持条件分支（如根据 EtherType 字段决定下一层协议的解析方式），但不支持循环——每个数据包的解析路径必须是有向无环的。解析器的状态数和转换规则受硬件资源限制，典型 PISA 芯片支持的最大解析深度约为 ~1000 Bytes。

（2）入口匹配-动作流水线（Ingress Match-Action Pipeline）。由 S_{ing} 个（典型值 12~20）MAU 级（Stage）级联组成 [14, 27]。在每个 MAU 级 s（$1 \leq s \leq S_{\text{ing}}$），数据包的 PHV 中的指定字段被用作匹配键（Match Key），在该级的匹配表（Match Table，实现为 TCAM 或 SRAM 哈希表）中进行查找。匹配成功后，执行对应的动作（Action），包括修改 PHV 字段、读写有状态存储元素（下文详述）、设置出口端口等。每个 MAU 级的处理在单个时钟周期内完成，且各级之间严格串行，不存在回路或反馈——这是 PISA 实现线速处理的根本约束。

（3）出口匹配-动作流水线（Egress Match-Action Pipeline）。结构与入口流水线类似，由 S_{egr} 个 MAU 级组成，在数据包经过排队调度（Queuing and Scheduling）后执行。出口流水线通常用于数据包的最终修改（如更新校验和、添加遥测字段）。

（4）逆解析器（Deparser）。将经过流水线处理后的 PHV 重新序列化为比特流，写入数据包的报文头区域，完成数据包的重组后发送至物理端口。

PISA 架构的核心约束可总结为：**无循环（No Loop）**——数据包在流水线中严格单向前进，不可回退；**固定时延（Deterministic Latency）**——每个数据包的处理时间恒定，不依赖于包内容或流状态；**有限级数（Bounded Stages）**——可用的 MAU 级数有限，复杂的处理逻辑需在有限级数内完成 [14, 27]。这些约束直接影响了第三章中多签名提取算法的流水线映射设计。

2.3.3 数据面有状态处理与存储约束

在 PISA 流水线中实现安全检测需要维护跨包的统计信息，这依赖于 P4 提供的有状态存储原语（Stateful Storage Primitives）。P4 定义了三类基本有状态元素 [14, 29]：

Register：可在 MAU 的 ALU 中被读取和更新的整数数组。Register 是 P4 中最灵活的有状态存储单元，支持读-修改-写（Read-Modify-Write, RMW）操作，如递增、赋值、条件更新等。但存在重要约束：同一个 Register 数组的同一个索引（Index）在同一个流水线通过中只能被一个 MAU 级访问，以避免读写竞争（Read-Write Hazard）。

Counter：仅支持递增操作的计数器，适用于统计匹配特定规则的数据包数量或字节数。

Meter：实现令牌桶（Token Bucket）算法的速率测量器，可对流量进行三色（Green/Yellow/Red）标记，用于速率限制和流量整形。

上述有状态元素均驻留在 PISA 芯片的片上 SRAM 中。如前文所述，宇航级芯片的片上 SRAM 容量严格受限，通常为数 MB 量级。假设检测算法需要维护 K 个状态条目（如 Top-k 候选流的多签名信息），每个条目占用 b 字节，则状态空间总消耗为 $M_{\text{state}} = K \times b\,\text{Bytes}$。以第三章的参数为例，若 $K = 1024, b = 128$（存储速率、FI/FO 位图、Persist 计数等），则 $M_{\text{state}} = 128\,\text{KB}$，可以舒适地容纳在宇航级芯片的 SRAM 预算内。这正是两级级联架构的设计优势：通过第一级的 Top-k 过滤，将需要进行精细多签名提取的流数量从全量（$\sim 10^6$）压缩至 K（$\sim 10^3$），从而将第二级的状态空间消耗控制在可接受范围内 [10]。

此外，PISA 的 ALU 所支持的运算类型严格受限于整数加减法、位移、按位逻辑运算和哈希计算。不支持浮点运算、乘法（部分高端芯片支持有限的硬件乘法器）、除法和索引间接寻址（Indirect Indexing）。这一计算约束要求所有在网算法的数学操作必须可被分解为上述基本操作的组合。例如，第三章中的可疑度评分函数需要避免使用除法和指数运算，转而采用基于位移的近似计算或预计算查找

表 [27]。

2.3.4 面向在网安全的近似数据结构

上述存储与计算约束使得精确的 Per-flow 状态维护在星载环境中不可行，驱动了近似数据结构（Approximate/Probabilistic Data Structures）在在网安全检测中的广泛应用。本文涉及的核心近似数据结构包括以下四种。

（1）布隆过滤器（Bloom Filter, BF）[30]。BF 由一个长度为 m 比特的位数组（Bit Array）和 k 个独立的哈希函数 $h_1, h_2, \ldots, h_k$ 组成。插入元素 x 时，将位数组中 $h_1(x), h_2(x), \ldots, h_k(x)$ 对应的位置置为 1。查询元素 y 时，检查 $h_1(y), \ldots, h_k(y)$ 对应的位是否全为 1。BF 不存在假阴性（False Negative），但存在假阳性（False Positive），假阳性率约为 $(1 - e^{-kn/m})^k$，其中 n 为已插入元素数量。在第三章中，BF 被用于 Fan-in/Fan-out 的基数近似估计：对于一条候选流，维护其源 IP 集合和目的 IP 集合的 BF，通过统计 BF 中被置位的比特数来近似估计集合的基数（利用 $\hat{n} = -(m/k)\ln(1 - X/m)$ 公式，其中 X 为被置位的比特数）。

（2）Count-Min Sketch（CMS）[31]。CMS 由 d 行 w 列的二维计数器矩阵组成，配合 d 个独立的哈希函数 $h_1, \ldots, h_d$。更新元素 x 的计数时，对每行 i，将 $\mathrm{CMS}[i][h_i(x)]$ 递增 1。查询 x 的频次时，返回 $\hat{f}(x) = \min_{1 \le i \le d} \mathrm{CMS}[i][h_i(x)]$。CMS 的估计具有单边误差性质：$\hat{f}(x) \ge f(x)$（始终高估），且 $\Pr[\hat{f}(x) - f(x) > \epsilon N] < \delta$，其中 N 为总元素数，$\epsilon = e/w, \delta = e^{-d}$。在第三章中，CMS 用作 Top-$k$ 重流检测的底层频次估计器。

（3）Space-Saving / Top-k 缓存结构 [32]。由一个容量为 K 的有序列表（Sorted List）维护当前观测到的 Top-K 重流。当新数据包到达时，若其所属流已在列表中，则更新其计数；否则，若列表未满，则插入新条目；若列表已满，则将新流的计数与列表中计数最小的条目比较，若超过则替换之。在 P4 流水线中，由于不支持循环遍历，Top-k 的比较与驱逐通常通过固定级数的串行比较（Cascaded Comparison）实现，每级处理一个候选位置。这一结构是第三章两级检测架构中第一级的核心组件。

（4）位图估计（Bitmap / Linear Counting）[33]。使用一个 m 比特的位数组，通过哈希函数将元素映射到对应位并置位。集合基数的估计基于空位数量 V：$\hat{n} = -m\ln(V/m)$。位图估计在精度可控的前提下将基数统计问题转化为简单的位操作，天然适配 P4 的 ALU 约束。在第三章中，位图被用于 Fan-in 和 Fan-out 的在线近似统计——对每个 Top-k 候选流维护独立的源 IP 位图和目的 IP 位图，每个数据包到达时仅需执行一次哈希计算和一次位置位操作。

2.4 分布式 Gossip 协议与协同防御基础

2.4.1 去中心化协同的必要性与传统共识的代价

第三章的单星检测方案解决了本地闭环响应的问题，但 LFA 攻击的碎片化特性使得任何单颗卫星只能观测到途经本节点的局部流量切片。在退化策略下，每颗卫星本地观测到的单流速率可能完全正常，但多颗卫星上的观测结果汇聚后才能揭示出异常的汇聚模式 [6, 10]。因此，多星之间的检测证据共享与协同判定是有效对抗 LFA 的必要条件。

直觉上，多节点协同可以通过集中式方案实现——即设置一个或多个中心协调节点，各卫星将本地检测结果上报至中心节点，由后者做出全局判定。但在 LEO 环境中，这一方案面临三个固有问题：（1）中心节点的指定与维护在高动态拓扑下本身就是一个非平凡问题（需要类似 Leader Election 的机制）；（2）中心节点到各卫星的通信路径可能跨越多个 ISL 跳数，引入额外时延；（3）中心节点本身可能是 LFA 攻击的目标，或其接入链路可能被攻击者瘫痪 [6]。

另一种方案是采用分布式强一致性共识协议（如 Paxos、Raft），使所有参与节点就攻击判定达成一致。但此类协议的通信复杂度和时延代价在 LEO 环境中不可接受：以 Raft 为例，一次共识需要至少两轮消息交换（提案-确认），涉及过半节点的参与。在 n 个节点组成的集群中，单次共识的消息复杂度为 $O(n)$，如果区域内有数十颗卫星参与协同，每次判定需要交换数十至数百条控制消息。在 ISL 带宽已被 LFA 攻击流量大量侵占的情况下，这些控制消息本身可能因拥塞而丢失或严重延迟，导致协议无法收敛。

全网广播泛洪（Flooding）是最简单的信息共享方式，但其消息复杂度为 $O(|V| \cdot |E|)$，在大规模星座（$|V| > 1000$）中将产生灾难性的信令风暴（Broadcast Storm），不仅耗尽剩余的可用带宽，还可能触发二次拥塞。

2.4.2 Gossip 协议的基本原理与传播特性

Gossip 协议（又称流行病协议, Epidemic Protocol） [34] 提供了一种介于强一致性共识与盲目泛洪之间的折中方案。其基本机制为：每个节点周期性地（周期为 T_{gossip}）随机选取一个或少数几个邻居节点，将本地持有的信息（如攻击检测证据）发送给对方；接收方将收到的信息与本地信息进行合并（Merge），并在后续周期中继续向其邻居传播。

Gossip 协议的传播动力学可类比为传染病模型中的 SIR（Susceptible-Infected-Recovered）模型。设网络中有 n 个节点，初始时有 i_0 个节点持有某条信息（”已感染”），每个已感染节点在每个 Gossip 周期中以概率 p 将信息传播给一个随机邻居。

在完全图（Complete Graph）上，信息到达所有节点所需的轮次为 $O(\log n)$，消息总量为 $O(n \log n)$。在稀疏图（如 LEO 的 4-regular mesh）上，传播速度取决于图的连通性和度数，但仍然保持对数级别的轮次复杂度 [34]。

Gossip 协议的关键优势在于：（1）**去中心化**——无需指定或维护中心协调节点；（2）**容错性**——即使部分节点或链路失效，信息仍可通过替代路径传播，最终到达所有可达节点（最终一致性, Eventual Consistency）；（3）**可扩展性**——每个节点在每个周期中仅与少量邻居通信，单节点的通信开销与网络规模无关。这些特性使 Gossip 协议天然适合在高动态、部分链路可能被攻击瘫痪的 LEO 环境中进行多星协同 [34]。

2.4.3 抑制型传播与开销控制

标准 Gossip 协议的一个固有问题是冗余消息的过度生成：已经获知某条信息的节点仍然可能反复收到相同信息的推送，造成带宽浪费。在 LFA 攻击已导致 ISL 带宽紧张的环境中，这种冗余尤其不可接受。

抑制型传播（Suppressed Broadcasting）机制通过引入冗余检测与主动静默来控制 Gossip 的开销。其核心思想源自物联网领域的 Trickle 算法 [35]，可形式化为以下流程。

对于每条待传播的消息 m，节点维护三个状态变量：传播计时器 T_m（在区间 $[T_{\min}, T_{\max}]$ 内动态调整）、冗余计数器 c_m（记录在当前周期内收到的与 m 内容一致的消息数量）、以及冗余抑制阈值 K。在每个传播周期开始时，节点将 c_m 重置为 0 并启动计时器 T_m。在计时器运行期间，每当节点从邻居收到一条与 m 内容一致的消息时，将 c_m 递增 1。当计时器到期时，若 $c_m < K$，节点判断其邻域内尚未充分获知该信息，遂主动向邻居发送 m；若 $c_m \geq K$，节点判断其邻域已充分获知该信息，遂抑制本次发送。若信息在一段时间内未再听到新的更新，T_m 倍增至 $T_{\max}$，节点进入静默状态 [35]。

此外，通过为每条消息设置 TTL（Time-To-Live）跳数限制，可将其传播范围限制在发起节点的 h-hop 邻域内（即感兴趣区域 Region of Interest, ROI），避免信息向无关区域扩散。在第四章的 CoopSatShield 设计中，ROI 的跳数半径 h 被设定为与 LFA 攻击路径的典型跳数相匹配，确保协同判定的参与节点恰好覆盖攻击路径经过的卫星群组 [10]。

2.5 本章小结

本章从四个维度构建了支撑后续研究的理论基础。在网络架构层面，建立了 LEO 巨型星座的时变图模型 $G(t) = (V(t), E(t))$ 和快照路由模型，明确了 ISL 容量约束 C_e 和拓扑切换周期 $\Delta\tau$ 的量化参数。在威胁模型层面，形式化描述了 Crossfire 型 LFA 的三阶段攻击流程和三种退化策略，论证了多维度特征检测的必要性。在计算平台层面，详细分析了 PISA 流水线的无循环、固定级数、有限 ALU 等硬约束，以及片上 SRAM 的容量上限，建立了在网算法设计的" 硬边界" 条件。在协同机制层面，引入了 Gossip 协议的传播理论与抑制型变种的开销控制模型，论证了其在 LEO 高动态、部分链路受损环境下的适用性。这些理论基础将在第三章（多签名在网检测）和第四章（多星协同推回）中被直接引用和具体化。

第三章 面向大规模低轨卫星网络的多签名在网检测与队列缓解方法

3.1 引言

大规模低轨卫星网络（Large-scale LEO Constellation Network, LSN）通过星间激光链路（Inter-Satellite Link, ISL）与星地链路为全球提供低时延、广覆盖的互联网接入。然而，与地面骨干网类似，LSN 作为高价值通信基础设施同样面临拒绝服务类攻击风险，其中以链路洪泛攻击（Link Flooding Attack, LFA）尤为突出。LFA 的典型策略是利用分布式受控终端（bots）发起大量看似正常的低速流量，将其沿着特定路径汇聚到目标链路，从而在链路上形成拥塞甚至阻断关键区域间通信。ICARUS 工作系统化刻画了面向 LSN 的 LFA 攻击面与攻击构造过程，指出攻击者可以借助公开轨道信息推测路由，并在拓扑动态变化下持续调整攻击流集合以维持链路拥塞，从而对 LSN 的可用性形成实质威胁 [5]。

从防御角度看，LSN 的网络形态决定了传统“控制面集中检测 + 下发策略”的闭环方案难以满足实时性。一方面，卫星网络的控制通道存在显著的带宽与时延限制，跨星/星地的控制闭环常达到毫秒到十毫秒量级；另一方面，攻击流集合可能在亚秒级甚至更短时间尺度内发生改变，使得控制面统计汇聚与策略更新的延迟会直接削弱防御有效性。SatShield 针对该矛盾提出利用可编程交换机将检测与缓解尽可能下沉到数据平面：以地址聚合速率作为区分特征，在数据平面持续维护 top-k 重流候选并通过队列调度实现被动丢弃，从而实现近实时的在网缓解 [10]。

尽管 SatShield 已证明“聚合速率”在现有卫星系统约束下具备良好的可分性，但仅依赖速率特征仍存在两个在工程与对抗层面都难以回避的问题。第一，真实网络中的正常业务存在长尾与大流现象（例如热门服务、突发上行等），仅用速率很难区分“合法大流”与“攻击聚合”；第二，攻击者可通过扩大 bot 规模、降低单 bot 速率或将攻击流量分摊到更多 decoy 目的来弱化按单一维度聚合后的速率可分性，从而使得基于 top-k 的候选生成质量下降，进而诱发漏检或误检。SatShield 在讨论部分也指出，随着 bot 规模与业务形态演进，引入 fan-in/fan-out 等结构性签名是提升鲁棒性的自然方向 [10]。地面网络中，Ripple [20] 和 Mew [21] 已展示了基于可编程数据面的 LFA 防御潜力，但它们依赖于地面交换芯片充裕的存储与计算资源，无法直接移植到 SWaP 受限的星载环境。

基于上述背景，本章在 SatShield 的“top-k 候选生成 + 队列调度缓解”的在网框架之上，提出一种多签名在网检测与缓解方法（MS-SatShield）。该方法以 top-k

为候选生成器，避免全量 distinct 统计带来的状态爆炸；同时在候选集合上引入 fan-in/fan-out 与持续性（persist）等结构性特征，实现对退化策略（多 decoy 分摊、脉冲式 on-off、速率下沉等）的鲁棒检测，并将多签名可疑度映射为队列优先级，实现对攻击的快速、低误伤缓解。

3.2 问题定义与方法设计

3.2.1 威胁模型与问题表述

将 LSN 在时刻 t 表示为动态图 $G(t) = (V, E(t))$，其中 V 为卫星节点集合，$E(t)$ 为随时间变化的星间链路集合。每条链路 $e \in E(t)$ 具有容量 C_e。在 LFA 场景中，攻击者控制一组分布式终端（bots）集合 $\mathcal{B}$，并选择一组 decoy 目的集合 $\mathcal{D}$，使得从 $\mathcal{B}$ 到 $\mathcal{D}$ 的攻击流量在转发路径上穿越目标链路 $e^\star$，从而使 $e^\star$ 的负载超过其容量并发生拥塞。与地面网络中常见的五元组拆分类似 [6]，攻击者可通过多对多配对（一个 bot 对多个目的、一个目的由多个 bot 访问）将单条五元组流的速率降低到接近正常水平，但这会显著改变流量的结构性特征，例如源的 fan-out、目的的 fan-in 以及可疑 key 的持续出现模式 [10]。

为了与可编程交换机的周期性统计与配置机制对齐，本章采用固定时间窗口（epoch）建模。记第 t 个 epoch 的时间长度为 Δ，在该窗口内对某类地址 key（源地址或目的地址）进行聚合统计。对任意 key a，定义其在 epoch t 内的聚合字节数与聚合速率分别为

$$\text{bytes}_t(a) = \sum_{p \in \mathcal{P}_t(a)} len(p), \qquad R_t(a) = \frac{\text{bytes}_t(a)}{\Delta}, \tag{3-1}$$

其中 $\mathcal{P}_t(a)$ 表示 epoch t 内所有满足“地址字段等于 a”的数据包集合，$len(p)$ 为包长。

为刻画结构性特征，记每个包还包含一个与 key 互补的对端字段 x：当以源地址作为 key 时，x 为目的地址；当以目的地址作为 key 时，x 为源地址。则可定义 key 的 distinct 对端规模：

$$F_t(a) = \left|\{x \mid (a, x) \text{ 在 epoch } t \text{ 内出现}\}\right|, \tag{3-2}$$

其中 $F_t(a)$ 在源 key 模式下对应 fan-out（FO），在目的 key 模式下对应 fan-in（FI）。

此外，考虑 LSN 拓扑与流量均具有动态性，单一窗口内的 top-k 结果可能出现抖动。为提升稳定性，引入持续性特征（persist），表示某 key 在最近 k_p 个 epoch

中进入候选集合的次数：

$$P_t(a) = \sum_{i=1}^{k_p} \mathbf{1}\left[a \in \mathcal{H}_{t-i}\right], \tag{3-3}$$

其中 $\mathcal{H}_t$ 表示 epoch t 导出的 top-k 候选 key 集合，$\mathbf{1}[\cdot]$ 为指示函数。

需要指出的是，速率退化与结构性特征放大之间存在定量约束关系。考虑 $|\mathcal{B}|$ 个 bot 将攻击流量均匀分摊到 M 个 decoy 目的的场景。在以目的地址为 key 的模式下，每个 decoy 的聚合速率为 $R_{\text{atk}} = A/M$，其中 $A = \sum_{b\in\mathcal{B}} r_b$ 为总攻击注入量（由式 (3-5) 约束）；同时每个 decoy 的 fan-in 为

$$\text{FI}_{\text{decoy}} = \frac{|\mathcal{B}|}{M} \cdot \eta, \tag{3-4}$$

其中 $\eta \in (0, 1]$ 为 bot 到 decoy 的配对覆盖率（均匀配对时 $\eta = 1$）。类似地，在以源地址为 key 的模式下，每个 bot 的 fan-out 为 $\text{FO}_{\text{bot}} = M \cdot \eta$。上式表明：当攻击者为降低 per-key 聚合速率而增大 M 时，FI 按 $|\mathcal{B}|/M$ 缩小但仍随 $|\mathcal{B}|$ 线性增长；而当攻击者扩大 $|\mathcal{B}|$ 以压低单 bot 速率 $r_b = A/|\mathcal{B}|$ 时，FI 同步放大。攻击者无法同时压缩速率与 FI/FO：若极端地令每个 bot 仅访问一个 decoy（$\eta \to 1/M$, FO=1），则每个 decoy 的 FI $= |\mathcal{B}|/M$ 仍可随 $|\mathcal{B}|$ 增长而保持可观（例如 $|\mathcal{B}| = 2000, M = 100$ 时 FI$= 20$，远高于良性 key 的典型 FI $1 \sim 5$），为多签名检测提供了理论抓手。

在本章后续实验中，除非另行说明，均以目的地址作为 key，从而 $F_t(a)$ 对应 fan-in（FI），衡量访问同一目的地的不同源地址数量。

本章的核心问题可表述为：在星载可编程交换机的有限片上存储与流水线算力约束下，如何在每包线速处理的前提下，在线维护 $\mathcal{H}_t$ 并在其上提取 $R_t(a)$、$F_t(a)$、$P_t(a)$ 等多维特征，进而对可疑 key 赋予不同的队列优先级，实现对 LFA 的快速缓解，同时尽量降低对良性业务的误伤。

3.2.2 可分性分析与退化动机

SatShield 的关键观察是：在现有 LEO 系统约束下，攻击者要使目标 ISL 达到拥塞，必须注入接近链路容量量级的总攻击流量；而受单终端上行能力、单星可服务用户数等约束，攻击流在以源/目的地址聚合后仍会呈现 Mb/s 量级的重尾分布，从而与绝大多数良性聚合速率形成可分性 [10]。为了将该观察落到可复现实验，本章采用与 SatShield 一致的 LEO 仿真与流量构造思路：使用 Starlink Shell 1 星座拓扑（1584 颗卫星）与 ISL 容量 20 Gb/s 作为基本网络模型，良性背景流量由 CAIDA 匿名互联网 trace（CAIDA Anonymized Internet Traces 2018 数据集，采集自 Equinix-Chicago 监测点）经重放得到，并通过人口分布模型映射到城市对之间的通信需求（映射方法详见文献 [10] 第 IV-A 节）。需指出，该 trace 采集于 2018 年，

图 3-1 可分性随 decoy 数 M 退化的 CDF 对比

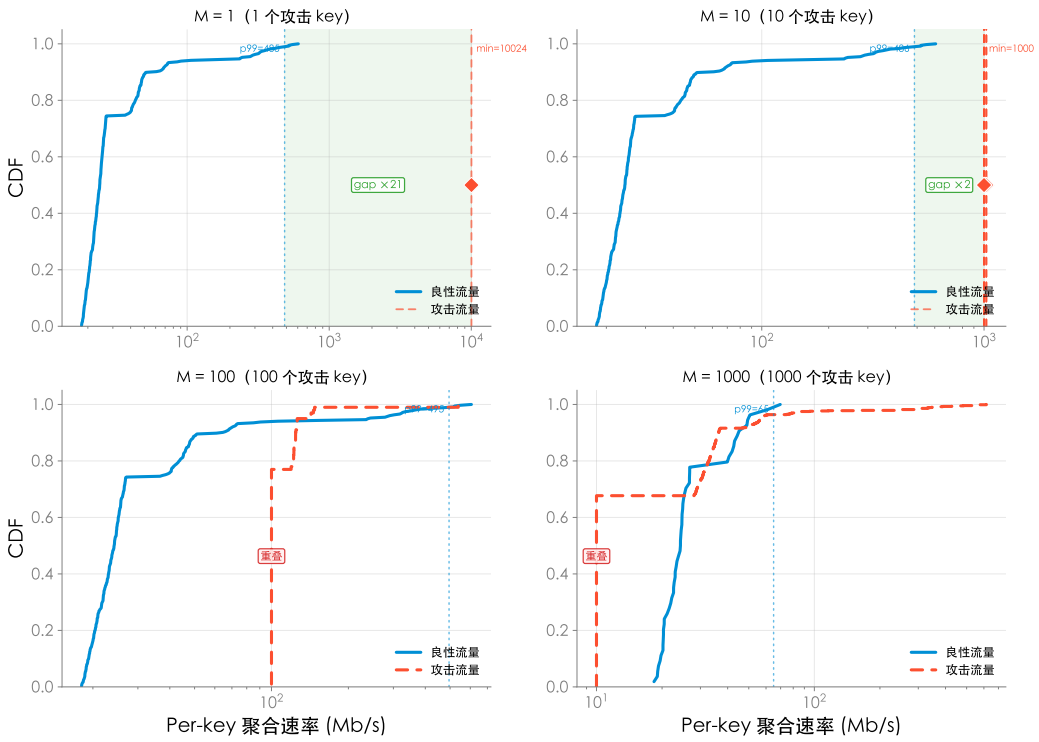


图 3-1 可分性随 decoy 数 M 退化的 CDF 对比。四个子图分别对应 $M = 1, 10, 100, 1000$，蓝色实线为良性 key 聚合速率 CDF，红色虚线为攻击 key 聚合速率 CDF（$M \leq 10$ 时因攻击 key 数量少，改用垂直线与散点标注）。绿色阴影区标注可分性间隙及倍数，红色“重叠”标注表示攻击与良性分布已交叉。

与当前互联网流量分布可能存在差异，但本文关注的是攻击侧与正常侧的相对可分性，良性分布的绝对形状对核心结论影响有限。

图 3-1 以 2×2 子图的形式展示了不同 decoy 数 M 下，良性与攻击流量在“每个地址 key 的聚合速率”维度上的经验分布函数（CDF）。具体绘制过程如下：首先在仿真中固定 epoch 长度 Δ，对每个 epoch 统计所有 key 的 $\text{bytes}_t(a)$，由式 (3-1) 得到 $R_t(a)$；随后分别对良性与攻击 key 集合绘制 CDF 曲线。当 $M = 1$ 时，良性 $p_{99} \approx 485$ Mb/s 与攻击聚合速率 10,024 Mb/s 之间存在约 21 倍的可分性间隙；$M = 10$ 时间隙缩小至约 2 倍；$M = 100$ 时攻击 key 速率下探至 100 Mb/s，已融入良性分布的长尾区间；$M = 1000$ 时攻击 key 与良性 key 完全重叠。该图直观展示了“可分性随 M 退化”的全过程，为后续多签名检测（图 3-3）提供了动机。

需要强调的是，图 3-1 的可分性并非“无条件成立”。当攻击者采取更强的隐蔽策略时，速率特征可能退化到与良性长尾区间发生重叠。为在论文层面明确方法的适用边界并给出可对齐的退化证据链，本章构造三类典型退化设定：其一为 bot

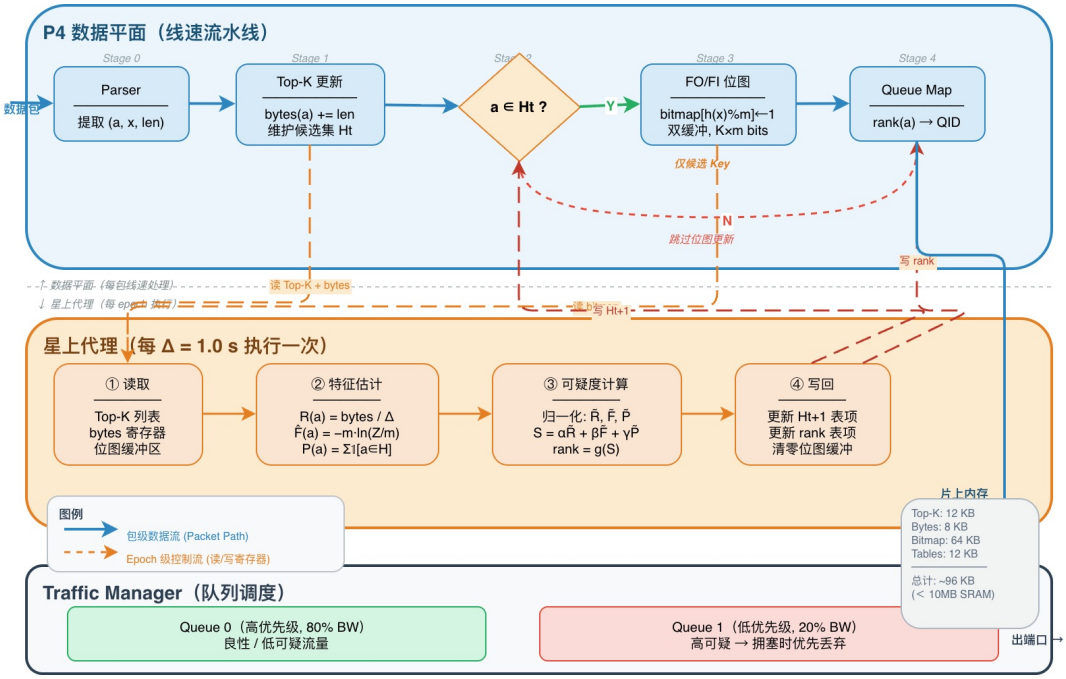


图 3-2 MS-SatShield 总体架构。数据平面以线速维护 top-k 候选并对候选 key 更新 FO/FI 位图；星上代理在每个 epoch 末读取候选统计，计算多签名可疑度并生成队列优先级映射表，写回交换机后由流量管理模块执行队列调度。

规模增大并降低单 bot 速率（速率下沉）；其二为将攻击流量分摊到更多 decoy 目的（多 decoy 分摊）；其三为采用脉冲式 on-off 策略使攻击 key 在窗口边界处“若隐若现”（引出持续性特征）。这些设定均满足 LFA 的基本约束：总攻击注入量需要与目标链路剩余容量同量级 [6]，即

$$\sum_{b\in\mathcal{B}} r_b \approx C_{e^\star} - U_{e^\star}^{\text{benign}}, \tag{3-5}$$

其中 r_b 为 bot b 的攻击发送速率，$U_{e^\star}^{\text{benign}}$ 为目标链路上良性占用带宽。式 (3-5) 直接说明：如果希望将每个 key 的聚合速率压低到“完全不可见”，必须相应扩大可用 bot 数量或增大 decoy 分摊规模。如式 (3-4) 所示，这将不可避免地放大 fan-in/fan-out 等结构性特征，为多签名检测提供抓手。

3.2.3 系统总览与数据流解释

图 3-2 展示了 MS-SatShield 的总体架构。系统由数据平面与星上代理（轻量控制平面）协同完成闭环：数据平面负责以线速维护 top-k 候选与轻量特征更新；星上代理以 epoch 为粒度拉取候选统计并计算多签名可疑度，将结果映射为队列优先级配置，再写回交换机用于下一轮调度。

为了便于读者从处理流程层面理解图 3-2，下面对” 包级数据流” 与”epoch 级控制流” 进行更细化的说明。需要首先明确时序关系：在 epoch t 的包处理中，系统使用 epoch $t-1$ 导出的候选集 $\mathcal{H}_{t-1}$ 和优先级映射 $\text{rank}_{t-1}(\cdot)$；当 epoch t 结束后，代理计算新的 $\mathcal{H}_t$ 与 $\text{rank}_t(\cdot)$ 并写回交换机，于 epoch $t+1$ 生效。

第一，包级处理在交换机流水线中按顺序发生。每个到达数据包 p 在解析后得到地址 key a、对端 x 与包长 len。系统首先更新 top-k 结构（本文采用 Space-Saving 算法 [32]，每包以 $O(1)$ 时间复杂度更新计数与排名）与聚合字节数寄存器 $\text{bytes}_t(a)$，该步骤对应“候选生成”，其目标是用固定内存近似维护当前窗口内的重 key 集合。随后，系统执行一次候选命中判断：若 $a \in \mathcal{H}_t$，则进入 FO/FI 近似统计更新模块，将 x 映射到位图索引并置位；若 $a \notin \mathcal{H}_t$，则跳过 FO/FI 更新以节省状态访问。最后，系统依据当前已安装的队列优先级映射表 $\text{rank}_t(\cdot)$ 选择队列并入队。上述顺序的关键在于：top-k 更新与 $\text{bytes}_t(a)$ 累加是所有包均需执行的基础操作，而 FO/FI 更新被严格限制在候选集上，从而将 distinct 统计的状态开销约束为 $O(K \cdot m)$。

第二，epoch 级控制流由星上代理触发。代理以固定周期读取 top-k 候选与字节计数，形成下一窗口候选集合 $\mathcal{H}_{t+1}$；同时读取候选 key 的位图，用线性计数（或 HLL-lite）估计 $\widehat{F}_t(a)$，并更新持续性 $P_{t+1}(a)$；随后将 $R_t(a)$、$\widehat{F}_t(a)$、$P_t(a)$ 归一化融合为可疑度 $S_t(a)$，再映射为队列优先级 $\text{rank}_{t+1}(a)$ 写回交换机。由于复杂运算（排序、归一化、对数/倒数等）均放在代理侧完成，数据平面只承担寄存器读写与简单算术，符合 PISA 流水线的可实现约束 [27]。

3.2.4 多签名可疑度建模与判决

MS-SatShield 的核心思想是将“候选生成”与“可疑判决”解耦：top-k 结构负责从海量 key 中筛选出可疑候选集合 $\mathcal{H}_t$，多签名模型负责在候选集合上区分攻击相关 key 与合法大流 key。为此，定义候选 key a 的综合可疑度为

$$S_t(a) = \alpha \cdot \widetilde{R}_t(a) + \beta \cdot \widetilde{F}_t(a) + \gamma \cdot \widetilde{P}_t(a), \tag{3-6}$$

其中 α, β, γ 为权重，$\widetilde{(\cdot)}$ 为归一化后的特征值。权重的设计遵循如下原则：α 与 β 为基础检测权重，满足 $\alpha + \beta = 1$，使得仅由 rate 与 FI/FO 组成的基础分数 $S_t^{\text{base}}(a) = \alpha\widetilde{R}_t(a) + \beta\widetilde{F}_t(a) \in [0,1]$；$\gamma$ 为持续性的加性奖励权重（additive bonus），独立于基础权重之外，其作用是为反复出现的可疑 key 提供额外加分。当 $\gamma = 0$ 时退化为双签名方案 S1；当 $\gamma > 0$ 时，$S_t(a)$ 的值域扩展为 $[0, 1+\gamma]$，但由于 γ 较小（本文取 $\gamma = 0.2$），阈值 $\tau = 0.5$ 的判决语义不受实质影响——τ 仍位于基础分数值域 $[0,1]$ 的中点，持续性仅为已超过 τ 的 key 提供额外置信度提升。本文默认取

$\alpha = \beta = 0.5, \gamma = 0.2$；消融实验（§3.3.5）将进一步分析权重选取对检测性能的影响。

归一化的目的在于使不同量纲、不同尺度的特征在同一分数空间可加。本文采用分位数归一化：以良性流量历史统计得到的分位点作为尺度基准，例如

$$\widetilde{R}_t(a) = \min\left(1, \frac{R_t(a)}{Q_{0.99}(R^{\text{benign}})}\right), \quad \widetilde{F}_t(a) = \min\left(1, \frac{\widehat{F}_t(a)}{Q_{0.99}(F^{\text{benign}})}\right), \tag{3-7}$$

其中 $Q_{0.99}(\cdot)$ 表示 0.99 分位点。$Q_{0.99}$ 可通过离线分析良性 trace 获得并作为配置参数写入代理；在线场景下亦可采用滑动窗口分位数估计，但需确保窗口长度覆盖多个 epoch 以避免被攻击流量污染。持续性 $\widetilde{P}_t(a)$ 可直接按窗口长度 k_p 归一化为 $\widetilde{P}_t(a) = P_t(a)/k_p$。

在做出“检测结果”之前，系统还需要将分数空间映射到队列优先级空间。设交换机支持 Q 个队列（或 Q 个优先级等级），定义一个单调映射函数 $g(\cdot)$：

$$\text{rank}_t(a) = g(S_t(a)) \in \{0, 1, \ldots, Q-1\}, \tag{3-8}$$

其中较小的 rank 表示较高优先级。本文采用分段线性映射，具体定义为

$$g(S_t(a)) = \begin{cases} 0, & \text{if } S_t(a) < \tau, \\ \left\lfloor \dfrac{S_t(a) - \tau}{S_{\max} - \tau} \cdot (Q-1) \right\rfloor + 1, & \text{if } S_t(a) \geq \tau, \end{cases} \tag{3-9}$$

其中 $S_{\max} = 1 + \gamma$ 为可疑度的理论上界（$\gamma = 0$ 时 $S_{\max} = 1$），τ 为检测阈值。当 $S_t(a) < \tau$ 时分配到最高优先级队列（rank $= 0$）；当 $S_t(a) \geq \tau$ 时按可疑度的相对位置均匀映射到 $Q-1$ 个低优先级队列，实现分级惩罚。本文实验中取 $Q = 4$, $\tau = 0.5$（参见表 3-1）。

3.2.5 FO/FI 在网近似统计结构

在网环境中对每个 key 精确维护式 (3-2) 的 distinct 计数会导致状态空间与更新开销不可接受。为保持可实现性，MS-SatShield 仅对候选集合 $\mathcal{H}_t$ 维护 FO/FI 的近似统计结构。本章以位图（bitmap）作为主线实现 [33]，原因在于其数据平面更新简单（置位操作）且内存预算可直接计算。

对每个候选 key a，分配一个长度为 m 的位图 $\text{bitmap}_t(a) \in \{0,1\}^m$。每当观察到关系对 (a, x)，计算

$$i = h(x) \bmod m, \qquad \text{bitmap}_t(a)[i] \leftarrow 1, \tag{3-10}$$

其中 $h(\cdot)$ 为哈希函数。epoch 末在星上代理侧读取位图，令 Z 为位图中 0 的个数，

算法 3-1: MS-SatShield 多签名在网检测与队列调度流程

数据平面（逐包处理）：
解析得到地址 key a（源地址或目的地址）、对端地址 x（目的地址或源地址）以及包长 len
更新 top-k 过滤器（以 (a, len) 更新计数）并累加 $\text{bytes}_t(a) \leftarrow \text{bytes}_t(a) + len$
if $a \in \mathcal{H}_t$ **then**
  计算 $i = h(x) \bmod m$，并置位 $\text{bitmap}_t(a)[i] \leftarrow 1$
根据当前映射表 $\text{rank}_t(\cdot)$ 确定队列优先级并入队

星上代理/控制平面（每个 epoch）：
从数据平面读出 top-k 列表与 $\text{bytes}_t(\cdot)$，形成候选集合 $\mathcal{H}_{t+1}$
if $\gamma > 0$ **then**
  将 $\{a \mid P_t(a) \geq 2,\ a \notin \mathcal{H}_{t+1}\}$ 合并到评分候选集 $\mathcal{H}_{t+1}$（持续性扩展）
读取 $\mathcal{H}_t$ 的位图并由式 (3-11) 估计 $\widehat{F}_t(\cdot)$，更新持续性 $P_{t+1}(\cdot)$
由式 (3-1)–(3-6) 计算多签名可疑度 $S_t(\cdot)$ 并由式 (3-9) 映射为队列优先级 $\text{rank}_{t+1}(\cdot)$
将 $\mathcal{H}_{t+1}$ 与 $\text{rank}_{t+1}(\cdot)$ 写回交换机表项
切换/清理位图缓冲区以开始下一 epoch

则可用线性计数 [33] 估计 distinct 基数：

$$\widehat{F}_t(a) = -m \cdot \ln\left(\frac{Z}{m}\right). \tag{3-11}$$

式 (3-11) 的计算包含对数运算，因此由代理完成，数据平面仅负责置位更新。为了避免每个 epoch 全量清零位图引入额外开销，同时降低攻击者" 对齐窗口边界" 的利用空间，本文实现采用双缓冲位图：奇偶 epoch 交替写入不同缓冲区，在读取后由代理对上一缓冲区执行批量清零，从而使数据平面无需执行大规模清理操作。具体而言，代理在每个 epoch 末通过控制平面 API 翻转一个单比特寄存器 `epoch_parity`，数据平面以该寄存器值选择写入的位图实例，从而无需流水线自行判断窗口切换。

需要指出的是，位图结构的误差会随着真实基数接近或超过 m 而快速增大（饱和）。然而在本章场景中，FO/FI 的作用并非精确估计超大规模基数，而是用作区分“正常较小 FO/FI”与“攻击导致的显著放大 FO/FI”。因此位图在有限内存下依然能够作为有效的判别签名。对于需要覆盖更大 FO/FI 范围的扩展需求，可采用 HLL-lite 结构提高抗饱和能力，本文在实验部分以消融方式给出对比（见 3.3.5）。

3.2.6 多签名在网检测与队列调度流程

算法 3-1 给出了 MS-SatShield 的整体流程。该流程与图 3-2 一一对应，并明确区分了数据平面逐包处理与星上代理逐 epoch 更新。

3.3 实验设置与结果分析

3.3.1 实验平台与基线方案

为验证 MS-SatShield 的检测与缓解效果，本文搭建了“LEO 攻击仿真 + P4 在网处理 + 流量重放评估”的实验框架。LEO 层面采用 ICARUS/SatShield 一致的仿真参数与星座模型（基于 Hypatia [26] 仿真框架），以保证攻击与背景流量分布具备可对齐的现实依据 [5, 10]；数据平面层面以 P4 [29] 实现 top-k 候选生成、候选位图更新与队列映射；星上代理在每个 epoch 周期性读取寄存器并下发队列优先级映射。基线对比方案主要包括：

（1）SatShield（rate-only，记为 S0）：仅使用归一化聚合速率 $\widetilde{R}_t(a)$ 作为可疑度（即 $\alpha = 1, \beta = \gamma = 0$），通过阈值 $\tau = 0.5$ 进行二元判决。该基线是 SatShield 原文 [10] 在本文框架内的等效实现，保留了其”top-k 候选 + 速率阈值判决” 的核心逻辑；

（2）MS-SatShield（rate + FO/FI，记为 S1）：在候选集合上加入 $\widehat{F}_t(a)$，取 $\alpha = \beta = 0.5, \gamma = 0$；

（3）MS-SatShield（rate + FO/FI + persist，记为 S2）：进一步加入持续性 $P_t(a)$，取 $\alpha = \beta = 0.5, \gamma = 0.2$，用于对抗脉冲式攻击与候选抖动。

三种方案均使用相同阈值 $\tau = 0.5$。由于 S0 与 S1 的可疑度值域均为 $[0, 1]$，$\tau = 0.5$ 的判决语义等价；S2 的 $\gamma = 0.2$ 为加性奖励（见 §第三章.2.4），不改变基础判决逻辑，因此共享 τ 的比较是公平的。上述对比对应后续消融实验的三个层级，使得”退化证伪基线—引入新签名恢复性能” 的证据链完整闭合。

3.3.2 数据集、流量生成与关键参数

良性背景流量采用 CAIDA 匿名 trace 进行重放，并依据人口（Pop）分布模型映射为城市对之间的通信需求；同时保证背景流量不会使任意链路长期超过 90% 利用率，以避免“未攻击已拥塞”的非预期干扰 [10]。攻击流量由 LFA 生成器构造：在每个攻击实验中随机选取城市对作为受害通信对，选择目标链路 $e^\star$ 并生成满足式 (3-5) 的攻击注入速率。表 3-1 总结了主要的网络与终端参数。

为了系统展示“速率特征退化—多签名恢复”的过程，本文在保持总攻击注入量 A 基本不变的前提下，构造多 decoy 分摊的退化设定。表 3-2 给出了该设定的参数组。表中包含 $M = 1$ 的退化下界情形，其目的并非声称真实攻击会只选择一个 decoy，而是作为对照用来展示“当每个目的聚合速率极大时，rate-only 必然有效”；随 M 增大，单 decoy 的聚合速率约为 A/M 并逐步逼近正常业务长尾区间，从而使 rate-only 的候选质量下降并引出 FI/FO 的必要性。

表 3-1 实验主要参数设置（与 SatShield/ICARUS 对齐）

参数	取值
星座拓扑	Starlink Shell 1（1584 颗卫星）
ISL 容量 C_e	20 Gb/s
单终端上行/下行峰值	64 Mb/s / 384 Mb/s
单星最大服务终端数	2080
背景流量来源	CAIDA trace 重放并按 Pop 模型映射
epoch 长度 Δ	1.0 s
候选规模 K	$\mathrm{clip}(0.3 \cdot N_{\text{keys}},\ 50,\ 1000)$（自适应）①
位图长度 m	256 bits
持续窗口 k_p	5 epochs
可疑度阈值 τ	0.5
高优先级带宽比	0.8（80% 带宽保留给良性/低可疑流量）

表 3-2 退化设定：多 decoy 分摊（降低按目的聚合速率，放大 fan-in/fan-out）

组别	$\lvert\mathcal{B}\rvert$	r (Mb/s)	A (Gb/s)	M	约单 decoy 聚合速率 A/M
B1	2000	5	10	1	10 Gb/s
B2	2000	5	10	10	1 Gb/s
B3	2000	5	10	100	100 Mb/s
B4	2000	5	10	1000	10 Mb/s

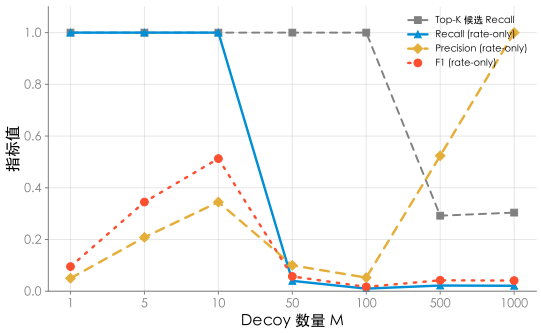
3.3.3 评价指标与绘制方式

检测侧采用 Precision、Recall、F1 作为核心指标。令真实攻击相关 key 集合为 F，算法输出集合为 $\widehat{F}$，则

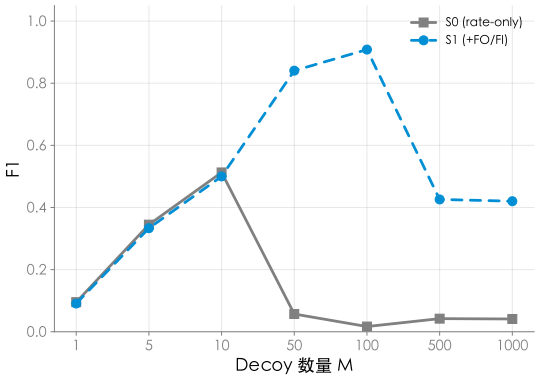
$$\text{Precision} = \frac{|F \cap \widehat{F}|}{|\widehat{F}|}, \qquad \text{Recall} = \frac{|F \cap \widehat{F}|}{|F|}, \qquad \text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (3\text{-}12)$$

其中 F 的定义需与攻击生成器一致：若以源地址为 key，则 F 定义为 bot 源地址集合；若以目的地址为 key，则 F 定义为 decoy 目的地址集合。缓解侧采用反应时间（reaction time）与良性吞吐下降（throughput drop）等指标，与 SatShield 的定义保持一致：reaction time 表示从观测到首个攻击包到系统开始生效缓解的时间间隔；throughput drop 表示攻击发生前后良性吞吐的下降比例 [10]。

图 3-3 的绘制遵循如下流程：对每组实验参数，首先利用仿真器与流量重放器生成包含标签的包序列；然后将包序列输入 P4 在网程序进行逐包处理与逐 epoch 统计；最后在离线评估脚本中按式 (3-12) 计算 Precision/Recall/F1 并绘制曲线。为了对齐 SatShield 的复现逻辑，本文同时给出“top-k 捕获准确性（对应 detector）”



(a) rate-only 检测器各指标随 M 变化



(b) S0(rate-only) vs S1(+FO/FI) 的 F1 对比

图 3-3 退化攻击下的检测性能。(a) 仅靠聚合速率（rate-only）的 top-k 候选 Recall、阈值判决 Recall/Precision/F1 随 decoy 数 M 的变化；(b) rate-only (S0) 与引入 FO/FI 后 (S1) 的 F1 对比，展示 FO/FI 特征在静态退化场景下的核心价值。ground truth 以 decoy 目的地址为 key。

与“攻击相关 key 识别准确性（对应多签名判决）”两类 F1 结果，从而将“候选生成质量”与“候选内判别能力”分离分析，避免将误差来源混淆。

3.3.4 结果分析：退化证据链与多签名收益

3.3.4.0.1 （1）可分性分析与 top-k 候选诊断 图 3-1 从分布层面验证了速率可分性随 M 退化的完整过程。$M = 1$ 时，良性 $p_{99} \approx 485$ Mb/s 与攻击聚合速率 10,024 Mb/s 之间存在约 21 倍间隙，top-k 候选可轻松捕获攻击 key。随着 M 增大到 100，攻击 key 的 per-key 速率降至 100 Mb/s，已完全融入良性长尾区间，速率特征失去区分力。

图 3-3(a) 从检测管线角度进一步诊断了 rate-only 方案的失效模式。该图同时给出 top-k 候选 Recall、阈值判决 Recall、Precision 与 F1 四条曲线。当 $M \leq 100$ 时 top-k 候选 Recall 保持 1.0，说明候选生成模块在中等退化下仍具备完整的攻击 key 捕获能力；然而阈值判决 Recall 在 $M = 50$ 时骤降至 0.04，F1 全程不超过 0.52，表明 rate-only 方案的瓶颈在于判决阈值无法区分攻击 key 与良性重流 key。当 $M \geq 500$ 时，top-k 候选 Recall 本身亦降至约 0.3，意味着大量攻击 key 的聚合速率已低于良性 key 而被 top-k 排除。需要说明的是，$M \geq 500$ 时 rate-only 的 Precision 出现回升（$M = 1000$ 时达 1.0），但这一现象源于 Recall 极低时（~ 0.02）判为正的 key 集合 $|\widehat{F}|$ 极小，分母缩减导致 Precision 的数值放大，并不反映检测能力的实质改善。

3.3.4.0.2 （2）FO/FI 在静态退化场景下的核心价值 图 3-3(b) 对比了 S0（rate-only, $\alpha = 1$）与 S1（+FO/FI, $\alpha = \beta = 0.5$）的 F1 随 M 的变化，展示了 FO/FI 特征在静态退化场景下的核心贡献。S1 的 F1 曲线呈现三阶段行为：

- **重叠段**（$M \leq 10$）：S0≈S1，两条线几乎重合。此时攻击 key 聚合速率 ≥ 1 Gb/s，rate-only 已能检出全部攻击 key，FO/FI 不提供额外增益。
- **分离段**（$M = 50 \sim 100$）：S1 大幅超越 S0。$M = 50$ 时 S0 F1 骤降至 0.057，而 S1 跃升至 0.84；$M = 100$ 时 S1 达峰值 0.908，S0 仅 0.017。其机理在于：虽然攻击 per-key 速率（$100 \sim 200$ Mb/s）已接近良性 p_{99}，但每个 decoy 的 fan-in（FI $= B/M = 20 \sim 40$）仍远高于良性 key 的典型 FI（$1 \sim 5$），FO/FI 特征提供了有效的判别能力。
- **衰减段**（$M \geq 500$）：S1 F1 下降至 0.42。攻击 per-key 速率降至 $20 \sim 35$ Mb/s，部分攻击 key 脱离 top-k 候选集（top-k Recall~ 0.3），候选集的覆盖率瓶颈成为 S1 F1 的天花板。此外，此时 FI$= |\mathcal{B}|/M = 2000/500 = 4$，已接近良性 key 的典型 FI 上界（$1 \sim 5$），FI 的区分能力同步退化，进一步限制了 S1 的判别空间。

需要指出的是，在所有 M 值下均存在约 20 个良性重流 key（rate=290 $\sim$ 606 Mb/s，FI=$13 \sim 26$）被误判为可疑。这些 key 代表真实的热门目的地，其 rate 与 FI 均处于良性分布的长尾区域，评分天然超过阈值 $\tau = 0.5$，构成了当前参数配置（$\tau = 0.5, \alpha = \beta = 0.5$）下基于 rate+FI 的检测器所观测到的误报基线。调整 τ 可改变该数值，但会相应影响 Recall（详见消融实验 §3.3.5）。在实际部署中，队列缓解是渐进式而非二元阻断，约 20 个被轻微降级服务质量的良性 key 是可接受的代价。

3.3.4.0.3 （3）持续性特征在动态攻击场景下的价值 图 3-3(b) 仅展示 S0 与 S1 的对比，未包含 S2（+persist）。其原因在于：在静态退化场景中，攻击 key 在每个 epoch 均进入 top-k，导致持续性 $\widetilde{P} \approx 1.0$；同时约 20 个良性重流 key 也几乎每个 epoch 进入候选集，$\widetilde{P}$ 同样趋近 1.0，持续性特征对两者加分相同，无法提供额外区分能力。

持续性的核心价值体现在*动态退化*场景中。图 **??** 展示了脉冲式 on-off 攻击（$M = 100$，on/off 各 10 epoch 即各 10 s，共运行 5 个完整 on-off 周期，总仿真时长 100 epoch）下 S0、S1、S2 三种方案的时域对比。结果呈现明显的层次分化：S0 在 $M = 100$ 下完全失效（on-phase 平均 Recall≈ 0.01），S1 通过 FO/FI 实现即时检测（Recall≈ 0.990），S2 进一步将 Recall 提升至 0.999。S2 的改进来源于两个机制：其一，采用加性奖励权重（$\alpha = \beta = 0.5, \gamma = 0.2$），即持续性为额外加分而非

从基础权重中借用（见式 (3-6) 的权重设计），保证 S2 的基础检测能力与 S1 完全相同；其二，当 $\gamma > 0$ 时，系统将历史上 $P \geq 2$ 但未进入当前 top-k 的 key 也纳入评分候选集（见算法 3-1 中的持续性扩展步骤），实现了对短暂跌出 top-k 的可疑 key 的” 记忆” 功能。关于遗忘效应：$k_p = 5$ 意味着在 off 持续 10 epoch 后，所有攻击 key 的 $P_t(a)$ 均衰减为 0，持续性分数完全归零，不会将历史攻击残留误带入下一个 on-phase 的判决中。在缓解层面，S0 的良性吞吐仅为 0.572，S1 恢复至 0.881，S2 进一步达到 0.885。

3.3.5 消融实验与参数敏感性分析

为进一步说明” 仅对 top-k 候选维护 FO/FI” 的资源—精度权衡，本节对候选规模 K、位图长度 m、epoch 长度 Δ 三个关键参数进行敏感性实验。实验固定退化设定为 $M = 100$（速率特征失效的分界点），采用 S1（$\alpha = \beta = 0.5$）方案进行评估。

3.3.5.0.1 （1）候选规模 K 图 3-4 展示了 Recall 与 F1 随 K 的变化。当 $K < 500$ 时，Recall 与 F1 几乎同步上升——此区间 K 过小导致大量攻击 key 落选候选集，是检测性能的瓶颈。K 增至 1000 时 Recall 饱和至 1.0，F1 达到约 0.91。继续增大 K（$2000 \sim 20000$）后 Recall 保持 1.0，F1 略有下降（趋近 0.90），原因是更多良性 key 涌入候选集增加了误报基数。从资源角度看，每个候选 key 需维护 $m = 256$ bit 位图，$K = 1000$ 时总内存约 $K \times m/8 = 32$ KB，在可编程交换机片上 SRAM（通常 $\geq$10 MB）中仅占极小比例。因此 $K \approx 1000$（或等效地 $K \approx 0.3 \cdot N_{\text{keys}}$）为推荐的默认值。

3.3.5.0.2 （2）位图长度 m 图 3-5 展示了 FO/FI 估计误差（MAPE）与总内存随 m 的变化。$m = 32$ 时 MAPE 约 4.4%，$m = 256$ 时降至 1.1%，$m = 1024$ 时进一步降至 0.5%。内存开销从 $m = 32$ 的约 20 KB 线性增长至 $m = 1024$ 的约 700 KB（$K = 1000$ 时）。综合考虑精度与开销，$m = 256$ 是帕累托最优点：MAPE 已降至 1.1%，足以区分攻击 FI（≥ 20）与良性 FI（$1 \sim 5$）；内存约 200 KB，处于可编程交换机可承受范围内。

3.3.5.0.3 （3）epoch 长度 Δ 图 3-6 展示了反应时间与平均良性吞吐下降随 Δ 的变化。$\Delta = 0.1$ s 时反应时间最短（约 0.2 s），但代理交互频率高达 10 次/秒，星上计算开销显著增加；$\Delta = 5$ s 时反应时间增至 10 s，对应良性吞吐下降约 24%。在 $\Delta = 0.5 \sim 1.0$ s 的推荐区间内，反应时间为 $0.5 \sim 1.0$ s（约 $1 \sim 2$ 个 epoch），良性

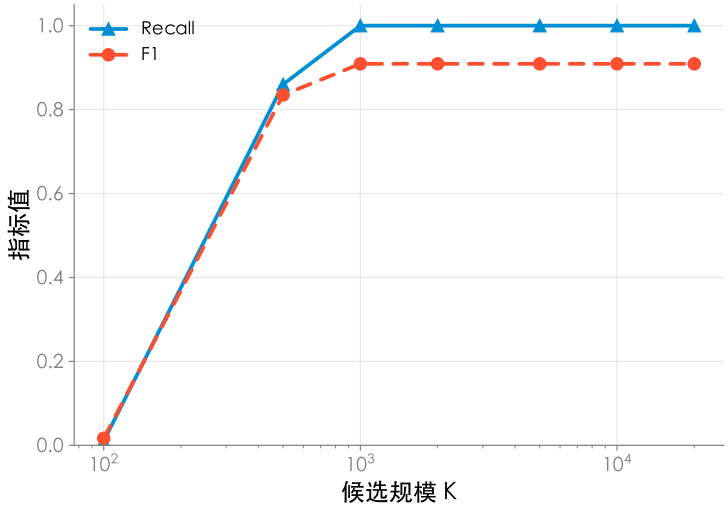


图 3-4 候选规模 K 对 Recall 与 F1 的影响（$M = 100$, S1 方案）。$K = 1000$ 时 Recall 饱和至 1.0，F1 达到约 0.91，为推荐的默认值。

吞吐下降约 12 ∼ 14%，系统在响应速度与计算开销之间取得较好平衡。本文选择 $\Delta = 1.0$ s 作为默认值。

需要指出的是，地面网络中已有若干针对 LFA 的检测与缓解方案。在 SDN 集中式路线上，Crossfire [6] 利用网络范围内的流量重布局与 SDN 集中控制来检测并解除链路拥塞，Coremelt [7] 则利用 NetFlow 分析识别可疑通信对，LinkScope [17] 通过主动探测检测目标链路拥塞，Woodpecker [18] 和 RADAR [19] 分别从路径重路由和相关性分析角度实现 SDN 驱动的 LFA 防御。在可编程数据面路线上，Ripple [20] 提出了基于 P4 的去中心化 LFA 防御，Mew [21] 进一步扩展了 P4 数据面的大规模 LFA 检测能力。然而，上述方案难以直接应用于 LEO 场景：SDN 集中式方案依赖低时延的控制通道与全局视图，而 LEO 的星地/星间控制链路存在显著的时延与带宽限制；P4 方案虽实现了数据面检测，但依赖地面交换芯片充裕的片上资源（如 Tofino 2 的 80 MB SRAM），无法满足星载设备的 SWaP 约束。MS-SatShield 的核心优势正在于将检测与缓解逻辑下沉至星载可编程交换机，以极低的状态开销（∼200 KB）实现近实时的在网缓解。

3.4 本章小结

本章针对大规模低轨卫星网络中的链路洪泛攻击，提出了 MS-SatShield 多签名在网检测与队列缓解方法。该方法在 SatShield 的 top-k 在网候选生成与队列调度框架上，引入 fan-in/fan-out 与持续性等结构性签名，实现了对速率特征退化场景的鲁棒识别；同时通过”仅对候选集合维护 FO/FI”的设计，将 distinct 统计的状态成本严格控制在 $O(K \cdot m)$（默认 $K = 1000, m = 256$，位图部分约 32 KB；加上

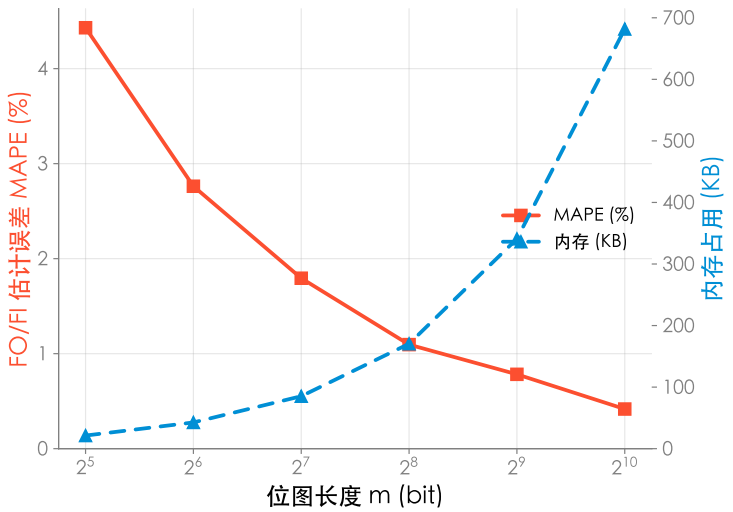


图 3-5 位图长度 m 对 FO/FI 估计误差（MAPE）与总内存的影响（$M = 100$, S1 方案）。$m = 256$ 时 MAPE 降至 1.1%，内存约 200 KB，为推荐的默认值。

top-k 计数器、字节寄存器与队列映射表等辅助结构，总数据平面状态约 200 KB）范围内。

实验结果表明：（1）在多 decoy 分摊退化下（$M = 100$），仅靠聚合速率的 S0 方案 F1 降至 0.017，而引入 FO/FI 后的 S1 方案 F1 恢复至 0.908，FO/FI 特征提供了核心检测增益；（2）在脉冲式 on-off 攻击下，持续性特征通过加性奖励与候选集扩展机制将 S2 方案的 on-phase 平均 Recall 提升至 0.999，良性吞吐恢复至 0.885；（3）消融实验确认了 $K \approx 1000$、$m = 256$、$\Delta = 1.0$ s 在检测精度与资源开销之间的帕累托最优性。

然而，上述结果仍局限于单星/单链路视角。当 $M \geq 500$ 时所有方案 F1 均下降至 $0.42 \sim 0.46$，受限于单链路 top-k 候选集的覆盖率瓶颈（top-k Recall~ 0.3）。此外，单节点缓解仅在拥塞已形成后的末端丢弃攻击流量，无法阻止攻击流在上游链路上的带宽浪费。这些局限性自然引出第四章的研究方向：通过多星协同的证据融合与逐跳推回机制，在网络级实现更早期、更彻底的攻击缓解。

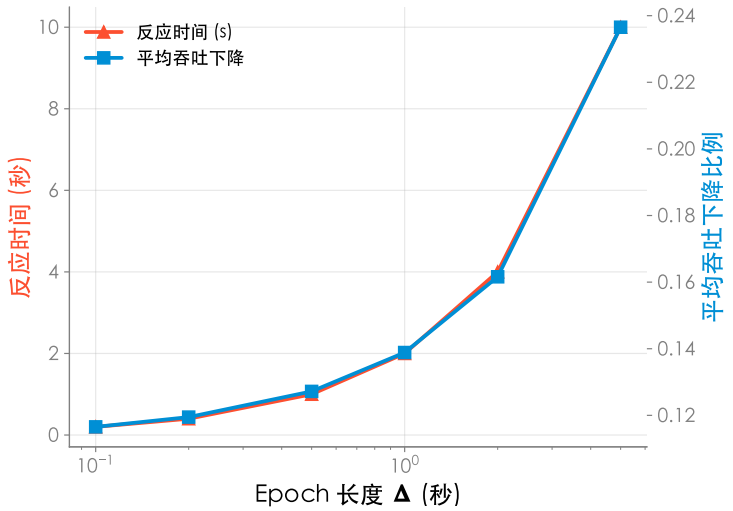


图 3-6 epoch 长度 Δ 对反应时间与良性吞吐下降的影响（$M = 100$, S1 方案）。$\Delta = 1.0$ s 时反应时间约 1 s，良性吞吐下降约 14%，为推荐的默认值。

第四章 面向大规模低轨卫星网络的多星协同在网计算与推回缓解方法研究

4.1 引言

上一章（第第三章章）围绕*单星/单节点*视角下的链路洪泛攻击（Link Flooding Attack, LFA）检测与缓解展开，利用可编程交换机的数据面线速能力实现了对可疑聚合 key 的实时识别与队列调度缓解。该类“星上本地闭环”的核心优势在于：避免依赖地面控制器的统计上报与策略下发，从而规避卫星-地面与星间控制链路带来的延迟开销。在 LEO 星座网络中，这一优势尤为关键：现有研究指出，卫星网络的控制环路延迟可达*几十毫秒*，而攻击流量可在*亚毫秒级*自适应变化，使得依赖中心控制器的传统 LFA 防御难以有效对抗快速演化的攻击 [10]。

然而，仅依赖“受害链路附近的本地缓解”仍存在两个根本性不足。第一，本地缓解通常发生在*拥塞已经形成*之后：攻击流量在到达目标瓶颈链路之前，已经在多条星间链路上消耗了大量带宽与队列资源。这种“末端丢弃”会在网络内部制造二次拥塞风险，并显著放大对良性业务的影响。第二，LEO-LFA 攻击具备强烈的*全网性与漂移性*：攻击者可以利用公开的轨道与拓扑信息，结合低时延路由偏好带来的路径可预测性，在全球范围灵活选择 bot 分布与攻击路径，从而使攻击证据以“碎片化”的形式分散在多个卫星节点上 [5]。在这种情况下，即使每个节点具备较强的本地检测能力，缺乏跨节点信息共享也可能导致整体*缓解*效力受限——各节点只能“就地丢弃”已到达本地的攻击流量，无法将抑制行动前推至更靠近攻击源头的上游节点，从而造成大量链路带宽的无效占用。

从物理世界的约束出发，LEO 星座网络天然具备支持“多星协同在网计算”的三点结构性条件。其一，典型星座具有*低度数*的星间连接结构，例如 Starlink Shell 1 中每颗卫星通常配置 4 条 ISL，ISL 容量可达 20 Gb/s 量级 [5]；其二，星座拓扑与最短路结构的变化发生在*秒级*时间尺度上，因而可以将系统视为一系列稳定的*拓扑快照*进行分时处理 [5]；其三，卫星节点逐步演进到“全栈可编程”架构，具备在数据面执行轻量计算与在星上 agent 执行局部控制的条件 [10]。基于此，本章提出一种面向大规模 LEO 的**抑制型邻居扩散协同（suppressed gossip）**与**推回缓解（pushback）**机制，在不依赖地面中心控制器的前提下，实现跨卫星的攻击证据聚合与靠近源头的分布式缓解，同时通过“区域限制 + 抑制扩散 + 速率配额”的三重机制避免在大规模网络中引发信令风暴。

本章的主要贡献体现在三个方面：首先，提出“本地证据抽取—协同证据聚

合一推回缓解执行”的多星协同在网计算框架，将上一章本地检测能力扩展为网络级威胁判断；其次，设计一种面向大规模星座的*抑制型邻居扩散协议*，在保持协同收敛速度的同时显著降低控制信令开销；最后，在 Starlink Shell 1 拓扑与真实流量背景下（CAIDA trace）完成系统实现与实验评估，展示相较 SatShield 等单星方案以及典型地面 LFA 防御基线（如 LinkScope、Woodpecker、RADAR、Ripple、Mew 等）在*反应时间*、*良性吞吐保护与控制开销*三方面的优势 [5, 10, 17–21]。

4.2 基于抑制型邻居扩散的多星协同在网计算方法

4.2.1 问题模型与设计目标

4.2.1.0.1 （1）时变星座与拓扑快照假设 将 LEO 星座表示为时变图 $G(t) = (V(t), E(t))$。由于卫星运动导致的最短路径变化发生在*秒级*时间尺度 [5]，本章采用离散 epoch 建模：在 epoch 长度 Δ 内，近似认为拓扑与路由策略不变，记为 G_τ（τ 为 epoch 索引）。该假设与 ICARUS 提出的“按快照预计算攻击流分配”的对手能力一致 [5]，也与 SatShield 在星上闭环配置 agent 的周期更新机制相兼容 [10]。

4.2.1.0.2 （2）威胁模型与推回收益 攻击者控制分布在全球范围的 bot 集合，生成看似合法的低速流量，并通过精心选择源-目的对使攻击流聚合在目标 ISL 上形成拥塞 [5, 6]。设目标瓶颈链路为 e，其容量为 C_e；攻击流量在该链路处的聚合速率为 A，则拥塞发生条件近似为 $B + A > C_e$（B 为背景流量）。若攻击流量在到达 e 前平均经过 h 跳 ISL，则仅在瓶颈处丢弃会导致攻击流在上游链路上造成的“无效占用”带宽约为：

$$W_{\text{waste}} \approx (h-1) \cdot A. \tag{4-1}$$

上述估算为上界估计，假设攻击流路径上各跳链路不重叠且无中间限速机制。实际网络中，由于路径共享与已有 QoS 策略，实测 W_{waste} 通常低于此上界；但作为量级分析工具，该近似足以揭示” 末端丢弃” 模式的代价规模。在 Starlink Shell 1 中，ISL 容量规模为 20 Gb/s[5]，而 ICARUS 与 SatShield 均表明攻击者在数百到数千 bot 规模下即可制造可观的攻击带宽 [5, 10]。因此，即使 h 取较小常数（实际由拓扑与源-目的分布决定），W_{waste} 仍可能达到数十 Gb/s 量级，从而触发更广泛的链路排队与连锁拥塞。这一推回收益的可量化性，是本章引入多星协同与 pushback 机制的直接动机。

4.2.1.0.3 （3）设计目标 在上述模型下，本章围绕三个根本目标设计系统：

第一，**快速性**：协同判断与 pushback 传播必须在攻击自适应时间尺度之前完

成闭环更新，避免中心控制器的高延迟反馈环 [10]。

第二，**可扩展性**：在星座规模达到千颗甚至万颗卫星时，协同机制必须避免“全网广播式”信令，控制开销应远小于 ISL 带宽，并保持数据面可实现。

第三，**稳健性**：面对分散化、低速化、脉冲化或滚动目标的攻击策略，协同机制应能够将碎片化证据聚合为网络级信号，降低本地 top-k 漏检对整体防御效果的影响（该问题在第第三章章已通过退化设定进行了论证）。

4.2.2 系统总体架构：CoopSatShield

为实现上述目标，本章提出多星协同在网计算系统 CoopSatShield。系统由三类组件构成：*本地证据抽取（Local Evidence）*、*抑制型邻居扩散协同（Suppressed Gossip）*与*推回缓解执行（Pushback Enforcement）*。其体系结构如图 4-1所示。

图 4-1左侧展示了单颗卫星节点 v 的内部架构，右侧展示了多节点协同拓扑。

单节点内部架构（图 4-1左栏）分为上下两层：下层为 P4 可编程交换机 [29]（数据面，每包线速处理），上层为星上 Agent（控制面，以 epoch 为周期）。

*数据面*包含 6 个核心模块，分为复用模块与新增模块两组：

（1）**复用模块**（Ch.3 MS-SatShield）：*top-K 候选生成*——维护候选 key 的聚合速率排序；*FO/FI 位图*——记录候选 key 的扇出/扇入结构特征；*队列调度*——基于 rank 执行优先级队列映射与限速。

（2）**新增模块**（$\star$ Ch.4）：*入端口贡献统计*——维护每个候选 key 在各入端口的字节统计 bytes(a,p)，用于后续 pushback 上游选择；*EG/PT 消息解析*——完成协同消息的轻量级 header 解析与 TTL 递减；*seen-set 去重*——以 Bloom Filter [30] 达成 O(1) 消息去重。

上述所有模块均可由可编程交换机的寄存器阵列与 match-action 表实现 [27]，每包处理保持 O(1) 复杂度。

控制面 Agent 包含 6 个模块：

（1）*多签名评分*（复用 Ch.3）——计算候选 key 的多维度可疑性评分；

（2）*证据缓存 $\mathcal{E}_{v,\tau}$*——存储 ROI 内接收到的 EG 消息；

（3）*抑制型扩散决策*——执行冗余抑制判断（$cnt \geq k$ 时抑制发送）；

（4）*协同聚合 S^{net}*——计算网络级加权评分（式(4-5)）；

（5）*Pushback 判定*——比较 S^{net} 与阈值 Θ_{pb} 触发推回；

（6）*EG 消息处理/合并*——执行 Aggregate 操作并构造聚合消息。

两层之间通过 epoch 读取/写回接口连接：Agent 每个 epoch 从 top-K 统计、FO/FI 位图与入端口贡献表中拉取数据，计算多签名评分后将 rank 及推回规则写回数据面。

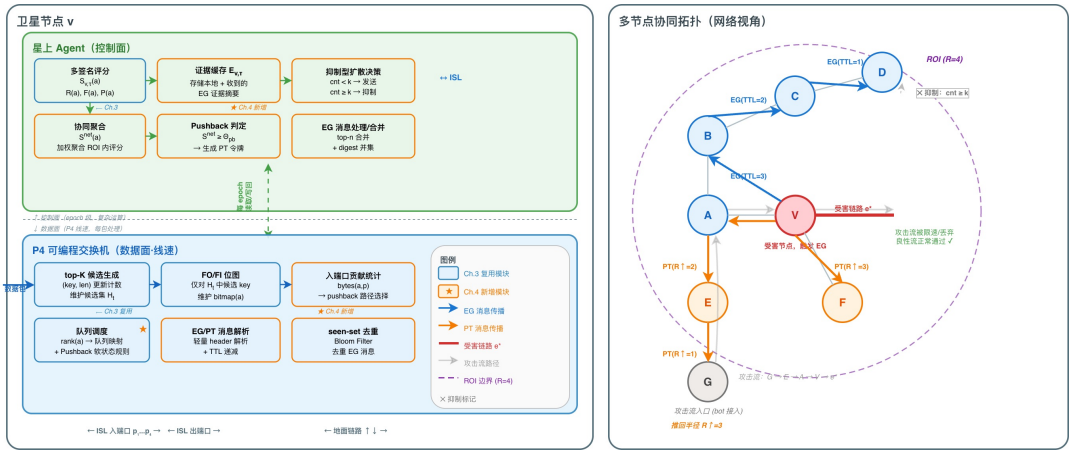


图 4-1 CoopSatShield 体系结构。左栏：单节点内部架构，上层为星上 Agent（控制面），下层为 P4 可编程交换机（数据面），标注 Ch.3 复用模块与 Ch.4 新增模块（⋆）。右栏：多节点协同拓扑，EG 消息向外扩散（蓝色）与 PT 令牌向上游推回（橙色），虚线圆标注 ROI 边界。

多节点协同拓扑（图 4-1右栏）展示了 7 颗卫星组成的局部拓扑。当节点 V 检测到受害链路 e^* 拥塞时，触发 EG 消息向外扩散（蓝色箭头，TTL 每跳递减），形成 ROI ($R = 4$) 范围内的证据共享与聚合；同时，PT 令牌沿贡献最大的上游方向反向传播（橙色箭头，$R_\uparrow$ 递减），沿途节点安装软状态缓解规则。右栏还标注了抑制机制的效果：当某节点在发送窗口内已监听到 k 条等价证据时，自动抑制转发（“× 抑制标记”），避免冗余广播。

4.2.3 本地证据抽取：从“单星检测”到“可协同的证据单元”

本章并不重新设计单星检测算法，而是将第第三章章的本地检测输出进一步结构化为“可协同的证据单元”。直观而言，协同的关键不在于每个节点输出一个硬判定，而在于输出*可聚合*、*可比较*、*可抑制传播*的证据摘要，以支持后续的低开销协同计算。

对卫星 v 在 epoch τ 内的某条出端口链路 $e = (v, u)$，定义其拥塞指示量为：

$$c_{v,e}(\tau) = \mathbf{1}\left[q_{v,e}(\tau) > Q_{\text{th}} \ \vee \ d_{v,e}(\tau) > D_{\text{th}}\right], \tag{4-2}$$

其中 $q_{v,e}(\tau)$ 表示该端口队列占用（或排队时延估计），$d_{v,e}(\tau)$ 表示端口丢包率（或 RED/ECN 触发率），$Q_{\text{th}}, D_{\text{th}}$ 为阈值。该指标完全来自数据面或交换机 traffic manager 可观测量，符合星上实时性要求。

当 $c_{v,e}(\tau) = 1$ 时，节点 v 从本地候选集 $\mathcal{H}_{v,\tau}$ 中提取与链路 e 最相关的 top-n 可疑 key 集合 $\mathcal{A}_{v,e,\tau}$，并为每个 $a \in \mathcal{A}_{v,e,\tau}$ 提供本地多签名评分 $S_{v,\tau}(a)$（见第第三章章式 (**??**)），同时输出该 key 在不同入端口的贡献分布 $\pi_{v,\tau}(a)$，用于后续 pushback

的上游选择：

$$\pi_{v,\tau}(a,p) = \frac{\text{bytes}_{v,\tau}(a,p)}{\sum_{p'} \text{bytes}_{v,\tau}(a,p')}, \tag{4-3}$$

其中 p 为 v 的入端口编号，$\text{bytes}_{v,\tau}(a,p)$ 为 epoch 内从端口 p 进入且匹配 key a 的字节数。由于 LEO 星间连接度低（典型为 4 条 ISL 加地面链路）[5]，该分布统计可在有限状态下实现，并可直接用于识别“攻击流主要来自哪个上游邻居”。

4.2.4 抑制型邻居扩散协同：避免信令风暴的 gossip 设计

（1）为何需要抑制机制

在大规模星座中，朴素 gossip（每个 epoch 每节点向所有邻居发送摘要并继续转发）会导致控制信令随网络规模线性增长。即使单条消息很小，当 $|V|$ 达到数千，持续的全网扩散仍会造成不必要的链路占用与处理负担，更严重的是会在拥塞期与数据业务争抢资源，形成“防御信令挤压业务”的反效果。LEO 网络的优势在于：攻击影响往往呈*局部区域性*（围绕受害链路与其上游汇聚路径），因此协同传播也应当局部化、事件驱动，并具备抑制重复传播的能力。

本章采用三重约束避免信令风暴：（i）**事件驱动**：仅当 $c_{v,e}(\tau) = 1$ 或接收到”新颖证据”时触发发送；（ii）**区域限制**：每条证据消息携带 TTL，最多传播 R 跳，形成受影响区域（Region of Interest, ROI）；（iii）**抑制扩散**：借鉴传感网中经典的冗余抑制思想 [34]（本章实现为 Trickle-like 机制），当节点在一个发送周期内已听到足够多重复证据时，自动抑制自身转发，从而避免爆炸式扩散。该抑制思想借鉴了 Trickle 协议 [35] 的冗余抑制原理，但与 Trickle 的倍增窗口机制不同，本章采用固定时隙 I 以简化星上实现（见下文）。

（2）协同消息类型与格式

本章定义两类星间控制消息：证据扩散消息（Evidence Gossip, EG）与推回令牌（Pushback Token, PT）。其中 EG 负责传播与聚合证据，PT 负责执行缓解推回。表 4-1给出 EG 消息字段。

其中 $\widehat{a_i}$ 为 key 的 CRC32 短哈希（32-bit），在实际 key 空间中碰撞率极低且节省带宽；$\widehat{s_i}$ 为量化评分（8-bit，线性映射 $[0,1]$ 至 $[0,255]$），分辨率 $1/256$ 足够排序与阈值判断。`digest` 采用 Bloom Filter [30] 编码候选 key 集合，接收端以 O(1) 代价判断本地候选是否与对方证据有重叠（即式(4-4)中 Ω 的快速近似），从而避免对无关证据执行昂贵合并。Bloom Filter 的位数 m 根据候选 key 数量 n 动态选取以保证可控的误报率：当 $n \leq 10$（典型 top-n 配置）时，$m = 128$ bit 已可保证误报率低于 2.5%（最优哈希函数数 $k = 9$ 时）；当候选集更大时，m 应相应增大（例如

表 4-1 Evidence Gossip（EG）消息字段设计

字段	含义
`epoch`	当前 epoch 编号，用于软状态一致性与过期清理
`link_id`	触发拥塞的链路标识（如 (v,u) 或端口号）
`ttl`	剩余传播跳数（ROI 半径 R）
`nonce`	消息唯一标识，用于去重缓存（seen-set）
`digest`	候选 key 集合摘要（例如 Bloom Filter），用于快速交集判断
$\langle \widehat{a}_i, \widehat{s}_i \rangle_{i=1}^{n}$	top-n 可疑 key 的哈希表示与量化评分
`cong_level`	拥塞强度（可由丢包率/队列占用量化得到）

表 4-2 Pushback Token（PT）消息字段设计

字段	含义
`key_hash`	目标 key 的 CRC32 短哈希 $\widehat{a}$（32-bit）
`rank`	缓解优先级（8-bit：High/Med/Low，映射到队列与限速）
`epoch`	触发 epoch 编号，用于软状态一致性
$R_{\uparrow}$	剩余推回跳数（4-bit），每跳递减直至 0 终止
T_{life}	软状态生存期（8-bit，以 epoch 为单位），到期自动清除

$n = 100$ 时需 $m \geq 1024$ bit 以维持 $< 1\%$ 的误报率）。该设计符合可编程数据面的常数时间操作约束，并有利于将部分处理逻辑下沉到交换机流水线实现。

图 4-2以 bit-level 字段图展示了两类消息的具体格式。EG 消息的固定头部仅 28 字节，可在 P4 交换机 parser 中以固定偏移完成线速解析；可变载荷部分每个 entry 占 5 字节（$\widehat{a}_i$ 32-bit + $\widehat{s}_i$ 8-bit），当 $n = 10$ 时 EG 消息总计 78 字节，$n = 100$ 时也仅 528 字节，远小于 ISL 链路的 MTU。PT 消息更为精简，固定仅 **9 字节**即包含目标 key 哈希 $\widehat{a}$（32-bit）、缓解优先级 `rank`（8-bit，映射到队列降权/限速强度）、剩余推回跳数 $R_{\uparrow}$（4-bit）与软状态生存期 T_{life}（8-bit，以 epoch 为单位），仅携带执行缓解所需的最少信息。图下方的 ISL 传播路径示意直观展示了 EG 的 TTL 递减外向扩散与 PT 的 $R_{\uparrow}$ 递减反向推回过程；右下角的消息大小对比条形图表明 PT 的开销不足 EG 的 $1/8$，协同通信极为轻量。

（3）抑制型扩散协议（Suppressed Gossip Protocol）

设节点 v 在 epoch τ 内维护一个“证据缓存”$\mathcal{E}_{v,\tau}$，存储在 ROI 内观察到的 EG 消息摘要。对任意 $(\tau, \texttt{link_id})$，节点以固定时隙 $I = \Delta/4$ 执行一次抑制型扩散（即每 epoch 内最多执行 4 轮扩散，$I \ll \Delta$ 保证在 epoch 结束前完成收敛）。其思想是：节点在 I 内随机选择一个发送时刻 t_{send}；在 t_{send} 前若已监听到至少 k 条“等价证据”（同 epoch 同 link_id 且 digest 高度相似），则抑制发送；否则在 t_{send} 发送

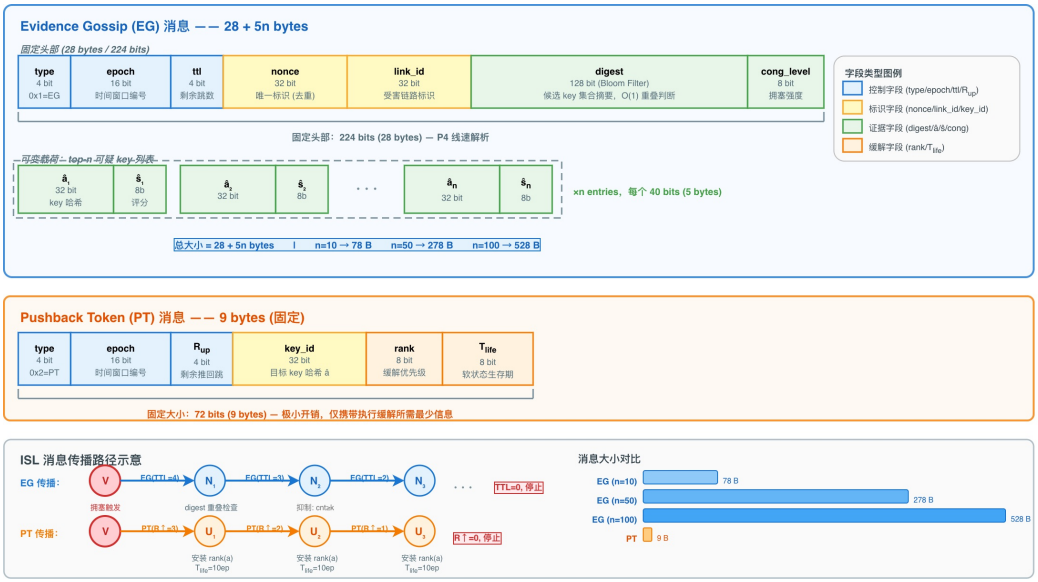


图 4-2 协同控制消息的头部格式。上部：Evidence Gossip（EG）消息，28 字节固定头部 $+5n$ 字节可变载荷，字段按功能类型配色（蓝色 = 控制、黄色 = 标识、绿色 = 证据、橙色 = 缓解）。中部：Pushback Token（PT）消息，仅 9 字节定长。下部：ISL 消息传播路径示意与消息大小对比。

聚合后的 EG 消息。通过随机化发送与冗余抑制，系统在保持传播速度的同时显著降低重复广播。

进一步地，为确保传播局部化，本章规定 EG 消息仅在满足“相关性门限”时才会被转发：节点 v 收到来自邻居 u 的 EG 消息 M 后，计算其与本地候选集合 $\mathcal{A}_{v,\tau}$ 的重叠度：

$$\Omega(M, v) = \frac{|\mathcal{A}_{v,\tau} \cap \mathcal{A}(M)|}{|\mathcal{A}(M)|}, \tag{4-4}$$

其中 $\mathcal{A}(M)$ 表示消息中 top-n key 集合。仅当 $\Omega(M, v) \geq \theta_\Omega$ 或本地链路也处于拥塞态 $c_{v,e}(\tau) = 1$ 时，节点才会将该证据纳入缓存并参与后续转发。该机制利用“本地观测的一致性”来过滤无关扩散，是避免信令风暴的关键。

算法 4-1给出了抑制型扩散协议的流程。与朴素 gossip 相比，其差异不在“传播能力”，而在于利用物理网络的局部性与低度数结构，将传播控制在 ROI 内，并用抑制机制避免同一证据在区域内反复回旋传播。

Aggregate 策略说明。函数 $\texttt{Aggregate}(\mathcal{E}_{v,\tau})$ 对缓存中的所有 EG 消息执行如下合并：（i）*评分取最大*：对同一 key a 出现在多条消息中的情况，取 $\hat{s}(a) = \max_j \hat{s}_j(a)$，以保留最强证据；（ii）*digest 按位 OR*：将各消息的 Bloom Filter 摘要按位取并集，保证接收端可判断聚合后候选集的完整覆盖；（iii）*截断*：合并后的 key 集合若超过 n，按 $\hat{s}(a)$ 降序保留前 n 个，确保消息大小恒定。该策略在保持消息紧凑的同

算法 4-1: 抑制型邻居扩散协同（Suppressed Gossip Protocol）

Input: epoch τ，ROI 半径 R，抑制参数 k，重叠阈值 θ_Ω

Trigger: 若存在链路 e 使 $c_{v,e}(\tau) = 1$，构造本地 EG 候选消息 M_v
将 M_v 加入缓存 $\mathcal{E}_{v,\tau}$，并进入抑制发送窗口 I
在窗口 I 内随机选择 t_{send}
while $t < t_{send}$ **do**
  监听邻居 EG 消息 M_u
  if M_u *未去重（nonce 未见过）且* $M_u.\texttt{ttl} > 0$ **then**
    计算 $\Omega(M_u, v)$（可由 digest 快速近似）
    if $\Omega(M_u, v) \geq \theta_\Omega$ ***or*** *本地存在拥塞链路* **then**
      将 M_u 合并入 $\mathcal{E}_{v,\tau}$（top-n 合并 +digest 并集）
统计窗口 I 内等价证据数量 cnt
if $cnt < k$ **then**
  生成聚合消息 $\widetilde{M}_v = \texttt{Aggregate}(\mathcal{E}_{v,\tau})$
  设置 $\widetilde{M}_v.\texttt{ttl} \leftarrow \min(\texttt{ttl}_{\text{received}} - 1, R)$，向所有邻居发送（不回发原入端口）
else
  抑制发送（suppress）

时优先传播高置信度证据。

TTL 收敛性保证。算法 4-1聚合发送步骤中 $\texttt{ttl} \leftarrow \min(\texttt{ttl}_{\text{received}} - 1, R)$ 确保每次转发 TTL 至少递减 1，即使经过聚合节点也不会重置为完整 TTL。因此，在 R 跳半径的 ROI 内，消息最多经历 R 次转发后必然终止，不存在无限传播风险。

4.2.5 协同判定与推回缓解：从“受害链路”推回到“上游入口”

（1）协同证据聚合与判定

在 ROI 内，多个节点对同一攻击相关 key 会给出不同程度的本地评分 $S_{v,\tau}(a)$。为形成网络级判定，本章定义协同聚合评分：

$$S_\tau^{\text{net}}(a) = \sum_{v \in \mathcal{R}_\tau} \omega_v \cdot \widetilde{S}_{v,\tau}(a), \tag{4-5}$$

其中 $\mathcal{R}_\tau$ 为在 epoch τ 内参与协同的 ROI 节点集合，$\widetilde{S}_{v,\tau}(a)$ 为归一化后的本地评分，ω_v 为权重。权重的设计根植于物理可观测量：拥塞越强、越接近受害链路的节点，其证据应更可靠。于是本章取：

$$\omega_v = \frac{(1 + \lambda \cdot \texttt{cong_level}_v) \cdot \exp(-\mu \cdot \texttt{dist}(v, e^{))}}{\sum_{u \in \mathcal{R}_\tau} (1 + \lambda \cdot \texttt{cong_level}_u) \cdot \exp(-\mu \cdot \texttt{dist}(u, e^{))}}, \tag{4-6}$$

其中 $\texttt{dist}(v, e)$ 表示 v 到受害链路 e 的跳数距离，λ, μ 为超参数，且所有参与节点的权重满足 $\sum_{v \in \mathcal{R}_\tau} \omega_v = 1$。$\lambda$ 控制拥塞等级对权重的放大程度，μ 控制距离衰减速率。在实现上，为避免复杂指数运算，控制面 agent 采用查表或分段线性近似完成权重计算；数据面只需提供 `cong_level` 与邻接关系。

推回判定阈值 Θ_{pb} 的选取遵循如下原则：在预设验证场景下（含 $M = 10, 50, 100$ 三组碎片化设定），绘制 ROC 曲线并在误推回率（即良性 key 被触发推回的概率）不超过 5% 的约束下选取使检测 F1 最大化的阈值，实验中取 $\Theta_{\text{pb}} = 0.7$。λ, μ 的取值通过小规模网格搜索确定（$\lambda \in \{0.5, 1.0, 2.0\}$，$\mu \in \{0.3, 0.5, 1.0\}$），初步调参表明 $\lambda = 1.0, \mu = 0.5$ 在 $M = 10, 50, 100$ 三组碎片化设定下均能取得稳定的 F1 和低误推回率。

当存在 a 使 $S_\tau^{\text{net}}(a) \geq \Theta_{\text{pb}}$ 时，系统对该 key 触发推回缓解；若多个 key 满足条件，则按 $S_\tau^{\text{net}}(a)$ 排序取前 n_{pb} 个执行，以保证数据面策略表规模可控。

（2）推回令牌（PT）与逐跳传播

pushback 的关键是：不依赖全局路由逆推，而是利用*逐跳可观测的上游贡献*实现“近似反向路径”传播。具体而言，受害链路附近节点 v 对 key a 计算其入端口贡献分布 $\pi_{v,\tau}(a, p)$（式(4-3)），选择贡献最大的前 $r = \min(2, d-1)$ 个上游端口集合 $\mathcal{P}_v^{\uparrow}(a)$（在 $d = 4$ 的 LEO ISL 结构中 $r = 2$），并向对应上游邻居发送 PT 令牌。令牌携带软状态生存期 T_{life} 与剩余推回跳数 $R_{\uparrow}$，上游节点收到后立即在本地数据面安装短时效缓解规则（如队列降权或限速），并重复上述过程继续向其上游传播。由于 LEO 网络连接度低且拓扑变化在秒级 [5]，该逐跳机制在每个 epoch 内具有稳定语义；同时通过软状态过期可避免拓扑变化后产生“幽灵规则”。

算法 4-2给出了 pushback 传播流程。

rank 到队列映射与限速动作。PT 中的 `rank` 字段映射到三级缓解强度：（i）$\texttt{rank} = \text{High}$：匹配 key 的流量被调度至最低优先级队列 Queue 0，带宽上限设为链路容量的 5%，实现近乎完全阻断；（ii）$\texttt{rank} = \text{Med}$：调度至 Queue 1，带宽上限为 20%；（iii）$\texttt{rank} = \text{Low}$：调度至 Queue 2，仅降低调度优先级但不硬限速。数据面通过 match-action 表项实现：以 key 哈希 $\hat{a}$ 匹配入向流量，命中后修改队列标记字段并施加对应 meter 速率。当同一 key 在同一 epoch 内收到多个 PT（来自不同下游节点或路径），取*最严格*的 rank 值（即 High > Med > Low）；若新 PT 的 rank 弱于已安装规则，则忽略。跨 epoch 时，新 PT 刷新 T_{life} 计时器，确保软状态的时效性。

图 4-3直观展示了上述推回过程的完整生命周期。该图分为三个阶段：

阶段 1（图左上角）展示受害节点 V 的拥塞检测过程：数据面检测到受害链路

算法 4-2: 逐跳推回缓解（Hop-by-hop Pushback）

Input: 触发 key 集合 $\mathcal{A}_\tau^{\text{pb}}$，推回半径 $R_\uparrow$，上游分支数 r

推回发起（节点 v）：

foreach $a \in \mathcal{A}_\tau^{pb}$ **do**

  在节点 v 生成 PT 令牌：$\langle a, \texttt{rank}, \tau, R_\uparrow, T_{\text{life}} \rangle$

  计算上游贡献分布 $\pi_{v,\tau}(a,p)$，选取前 r 大端口 $\mathcal{P}_v^\uparrow(a)$

  向 $\mathcal{P}_v^\uparrow(a)$ 对应邻居发送 PT

推回接收（节点 u 收到 PT）：

if *PT 未过期且* $R_\uparrow > 0$ **then**

  在本地数据面安装/刷新软状态规则：$\texttt{rank}(a) \leftarrow \text{PT.}\texttt{rank}$，有效期 T_{life}

  $R_\uparrow \leftarrow R_\uparrow - 1$

  基于 $\pi_{u,\tau}(a,\cdot)$ 继续向上游前 r 大邻居传播

e^* 的队列占用超过阈值（$q_{v,e^*} > Q_{\text{th}}$，即 $c(v,e^*) = 1$），并输出 top-K 候选 key 的入端口贡献分布 $\pi(a_1,p)$。图中小型条形图显示 key a_1 在三个入端口的贡献比例分别为 70%、20% 和 10%，指明主攻击方向。当协同聚合评分 $S^{\text{net}}(a_1) = 0.87 \geq \Theta_{\text{pb}} = 0.7$ 时，系统触发推回。

阶段 2（图中部）展示 PT 令牌的逐跳反向传播。受害节点 V 生成 PT$= \langle a_1, \texttt{rank} = \text{High}, \tau, R_\uparrow = 3, T_{\text{life}} = 10 \rangle$，选择贡献最大的前 $r = 2$ 个上游端口（p_1: 65% $\rightarrow U_1$，p_2: 20% $\rightarrow U_1'$），向对应邻居发送 PT。各接收节点（U_1, U_1'）立即在本地数据面安装软状态缓解规则 $\texttt{rank}(a_1) \leftarrow \text{High}$，并将 $R_\uparrow$ 递减后继续向上游传播。经过 3 跳后 $R_\uparrow = 0$，传播终止于节点 U_3, U_3'。图中金色 shield 图标标注了所有已安装缓解规则的节点，绿色渐变链路标注了推回释放的带宽。

阶段 3（图右下角）定量对比推回前后的效果：推回前攻击流经历 $h = 5$ 跳到达 e^*，$W_{\text{waste}} = \text{rate} \times 5$；推回后攻击流在 U_3 处被限速/丢弃（$h = 2$ 跳），$W_{\text{waste}} = \text{rate} \times 2$，降低 60%。图右上角的时间线展示了软状态生命周期 T_{life}：PT 在 epoch τ 安装后持续生效 $T_{\text{life}} = 10$ 个 epoch，到期自动清除；若攻击仍在持续，新一轮 PT 将刷新规则，避免拓扑变化后产生”幽灵规则”。

4.2.6 信令开销与可实现性分析

（1）ROI 限制下的信令规模上界

设星座网络的平均度数为 d（典型 LEO ISL 结构中 $d \approx 4$）[5]。对任意 EG 消息，TTL 为 R 时，其在最坏情况下可触达的节点数不超过一个 d 叉树的 R 层节点

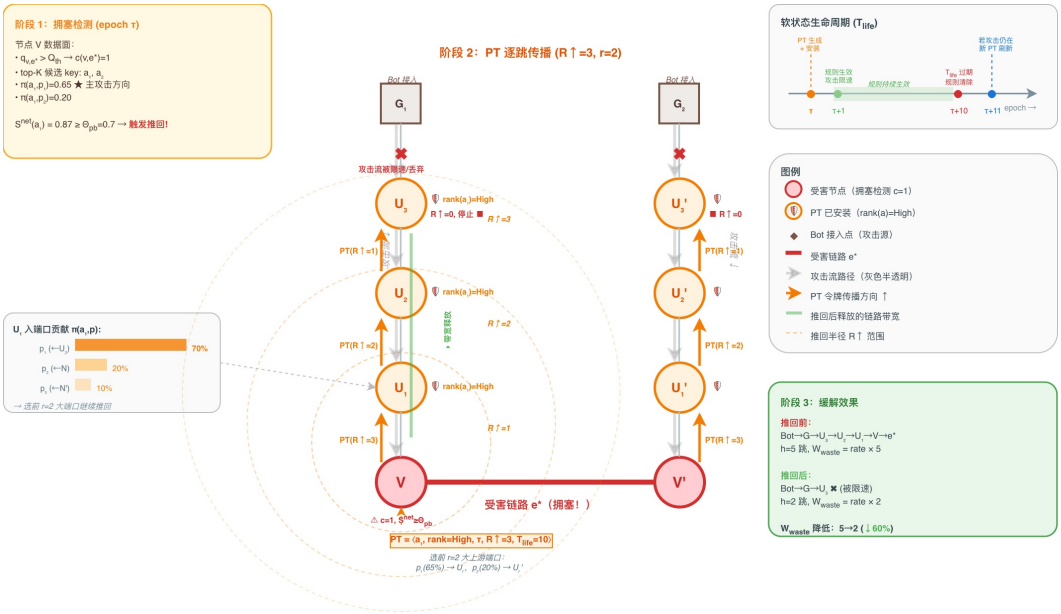


图 4-3 逐跳推回缓解传播示意。左上：阶段 1——拥塞检测与入端口贡献分布 $\pi(a,p)$；中部：阶段 2——PT 令牌从受害链路 e^* 向上游逐跳扩散（$R_\uparrow = 3$，$r = 2$ 分支），沿途节点安装软状态缓解规则；右下：阶段 3——推回前后的 W_{waste} 对比（h 从 5 跳降至 2 跳）。右上：软状态生命周期 T_{life} 时间线。

规模：

$$|\mathrm{ROI}(R)| \leq 1 + d \cdot \frac{(d-1)^R - 1}{d-2}, \quad d > 2. \tag{4-7}$$

当 $d = 4$ 时，上界为 $|\mathrm{ROI}(R)| \leq 2 \cdot 3^R - 1$。例如 $R = 4$ 时，最坏触达规模不超过 161 个节点，远小于 Starlink Shell 1 的 1584 节点总规模 [5]。因此，TTL 本身已将协同传播限制为“受影响局部区域”，避免全网泛洪。

（2）抑制机制下的消息量与带宽开销

在抑制型扩散中，节点在一个发送窗口内若监听到不少于 k 条等价证据即抑制发送，从而大幅降低重复广播。设 ROI 内有效参与节点数为 N_R，每个节点每 epoch 至多发送一次聚合 EG 消息（多余将被抑制），则 EG 消息总数满足：

$$M_{\text{EG}}(\tau) \leq N_R. \tag{4-8}$$

进一步地，设每条 EG 消息大小为 L_{EG} 字节，则控制带宽开销上界为：

$$B_{\text{EG}} \leq \frac{N_R \cdot L_{\text{EG}}}{\Delta}. \tag{4-9}$$

结合式(4-7)与 Starlink ISL 20 Gb/s 容量级 [5]，当 R 取小常数（本章默认 $R = 4$）、L_{EG} 处于 KB 级时，B_{EG} 相对 ISL 带宽占比可控制在远低于 1% 的量级。实验中我

们进一步验证了控制开销与 R, k, Δ 的敏感性（见第4.3.2节），并观察到抑制机制使得在攻击高峰期也不会出现信令风暴。

（3）数据面实现可行性

实现层面，本章遵循“数据面负责常数复杂度快路径，控制面负责聚合与决策”的原则。数据面新增逻辑主要包括：（i）为协同消息定义轻量 header 解析与 TTL 递减；（ii）维护 seen-set（可用 Bloom Filter 近似）用于消息去重；（iii）维护 key 的入端口贡献统计 $\text{bytes}_{v,\tau}(a, p)$；（iv）执行 pushback 规则（队列降权/限速）并以软状态过期机制清理。上述操作均可由可编程交换机的寄存器阵列与 match-action 表实现 [27]，且每包处理保持 O(1) 复杂度，满足线速约束。复杂的聚合评分（式(4-5)）与抑制决策在星上 agent 完成，从而避免数据面计算负担过重。

4.3 仿真与分析

4.3.1 实验设置

本章的仿真实验基于自研 Python 仿真框架实现（与第第三章章相同），该框架集成了星座拓扑构建、最短路计算、epoch 级事件驱动与数据面行为模拟，拓扑构建与路由策略参考 Hypatia 仿真平台 [26] 的快照路由模型 [4]。本章在 Starlink Shell 1 拓扑上进行评估，该拓扑包含 1584 颗卫星（72 轨道面 ×22 颗/面），ISL 为典型 4 链路连接结构，ISL 带宽 20 Gb/s，上下行链路容量为数 Gb/s 量级 [5]。epoch 长度 $\Delta = 1$ s，与卫星拓扑快照更新周期对齐 [5]。表 4-3汇总了本章实验的核心参数设置。为贴近真实网络背景流量，我们采用 CAIDA UCSD Anonymized Internet Traces 2018 数据集（OC-192 链路，采集于 Equinix-Chicago 监测点）构造良性流量：原始 trace 按比例缩放至 20 Gb/s ISL 容量级别，并按人口加权的城市对分布模型将源-目的 IP 映射到 LEO 地面终端对，具体映射参数与第第三章章实验设置一致 [10]。攻击流量生成采用 ICARUS 式 LFA 策略：攻击者利用公开轨道与拓扑信息选择 bot 对并计算使目标链路拥塞的流量分配 [5]；同时在更强对手模型下引入“低速化/多 decoy/脉冲化/滚动目标”等策略，以检验协同机制对碎片化证据聚合能力。

评价指标分为三类：（1）防御效果：良性吞吐、端到端时延、受害链路拥塞持续时间、通信可达性；（2）检测质量：攻击相关 key 识别的 Precision/Recall/F1（定义同第第三章章式 (**??**)）；（3）协同开销：EG/PT 消息数与字节开销、ROI 规模、收敛时间（从首个拥塞触发到 pushback 生效的时间），以及由推回带来的“无效带宽占用”下降比例（对应式(4-1)）。

表 4-3 第四章实验核心参数设置

参数	含义	取值
Δ	epoch 长度	1 s
I	抑制扩散时隙	$\Delta/4 = 0.25$ s
R	ROI 半径（EG TTL）	4 跳
$R_{\uparrow}$	推回半径（PT 跳数）	3 跳
k	抑制参数	3
n	top-n 候选 key 数	10
r	推回上游分支数	$\min(2, d-1) = 2$
θ_{Ω}	相关性门限	0.3
Θ_{pb}	推回判定阈值	0.7
λ	拥塞权重放大因子	1.0
μ	距离衰减因子	0.5
T_{life}	PT 软状态生存期	10 epoch

表 4-4 对比基线设置（第四章）

方案	说明
NoDefense	无防御，FIFO/ECMP 默认转发
SatShield	单星 top-k 过滤 + 队列调度缓解 [10]
MS-SatShield	第第三章章多签名增强，但无协同/无 pushback
LinkScope	基于探测/测量的 LFA 检测框架 [17]
Woodpecker	基于 SDN/TE 的 LFA 缓解（重路由/工程控制）[18]
RADAR	基于相关性分析的 LFA 检测与限速 [19]
Ripple	去中心化的可编程网络 LFA 防御框架 [20]
Mew	面向动态与资源受限环境的 LFA 防御机制 [21]
CoopSatShield	本章方案：抑制型扩散协同 + 逐跳 pushback

表 4-4给出对比基线。由于 LinkScope、Woodpecker、RADAR、Ripple 与 Mew 均面向地面网络，直接移植会因控制环延迟与拓扑快变导致不公平对比 [10]。本章在实现时保持各方案核心思想不变，并针对 LEO 环境进行如下适配：

- **LinkScope**[17]：原方案依赖周期性链路探测来推断拥塞位置。适配时将其探测周期对齐 epoch 长度 Δ，控制环延迟设为 ISL 单跳 RTT（约 $5 \sim 15$ ms）的倍数，以反映 LEO 星间通信的真实时延。
- **Woodpecker**[18]：原方案通过 SDN 控制器执行流量工程（TE）重路由。适配时在每个拓扑快照 G_{τ} 内独立执行 TE 优化，epoch 切换时基于新拓扑重新计算路径，控制器响应延迟设为 2Δ 以模拟 LEO 场景下 SDN 控制环的额外开销。

- **RADAR**[19]：原方案基于流量相关性分析检测 LFA。适配时将相关性分析窗口严格限制在单个 epoch Δ 内（不跨 epoch），避免拓扑变化导致的虚假相关。
- **Ripple**[20]：原方案假设去中心化可编程交换机网络，与 LEO 的 ISL 连接结构天然兼容。适配时保持其去中心化架构，将节点度数约束为 $d = 4$（对应 LEO ISL 结构），控制消息传播延迟按 ISL RTT 设置。
- **Mew**[21]：原方案面向动态与资源受限环境。适配时将其资源约束映射为 ISL 带宽上限（20 Gb/s）与星上计算能力限制，动态性映射为 epoch 级拓扑变化。

4.3.2 结果与分析

（1）推回缓解带来的网络级收益

图 4-4以分组柱状图的形式展示了在不同碎片化程度 $M \in \{1, 10, 100, 1000\}$ 下，各方案对" 无效带宽占用"W_{waste} 的影响。实验结果呈现清晰的三阶段趋势：

（i）**低退化段**（$M \leq 10$）：SatShield 与 MS-SatShield 表现相当，均可将 W_{waste} 降至 NoDefense 的 7.2%（$M = 1$）和 19.8%（$M = 10$）。此时攻击 key 聚合速率远高于良性长尾，rate-only 已能有效检出，因此多签名不提供额外增益。CoopSatShield 通过推回将丢弃点进一步前移至上游 $R_{\uparrow} = 3$ 跳处，W_{waste} 额外降至 3.0%（$M = 1$）。需要指出的是，$M = 1$ 时推回收益*纯粹来自 pushback 的路径缩短机制*而非协同证据聚合（此时单节点已足以检出唯一的攻击 key），这表明即使在无碎片化场景下，pushback 本身也具有独立的带宽节约价值。

（ii）**中退化段**（$M = 100$）：SatShield 的 rate-only 检测完全失效（F1= 0.017），$W_{\text{waste}} = 248{,}995$ Mb/s·hop，与 NoDefense 的 251,005 仅差 0.8%，几乎等于无防御。这一结果与第 第三章 章图 3-3(b) 的" 分离段" 分析一致：$M = 100$ 时速率特征已融入良性长尾。而 MS-SatShield 凭借 FO/FI 特征保持有效检测（F1= 0.909），W_{waste} 降至 NoDefense 的 17.1%；CoopSatShield 通过推回进一步降至 11.0%，相比 MS-SatShield 额外释放了 6.1 个百分点的链路带宽。

（iii）**高退化段**（$M = 1000$）：攻击 key 大量脱离 top-K 候选集（TopK Recall$\approx$ 0.30），所有方案 W_{waste} 均上升。MS-SatShield 与 CoopSatShield 仍优于 SatShield（占 NoDefense 的 72.5% 和 70.8%），但改善幅度有限，反映了第 第三章 章指出的候选集覆盖率瓶颈。

上述结果直接验证了式(4-1)的推回收益机理：推回将攻击流的平均承载跳数 h 从全路径长度压缩到 $R_{\uparrow}$ 跳级别。同时，多 M 值的渐进退化设定建立了从"rate-only

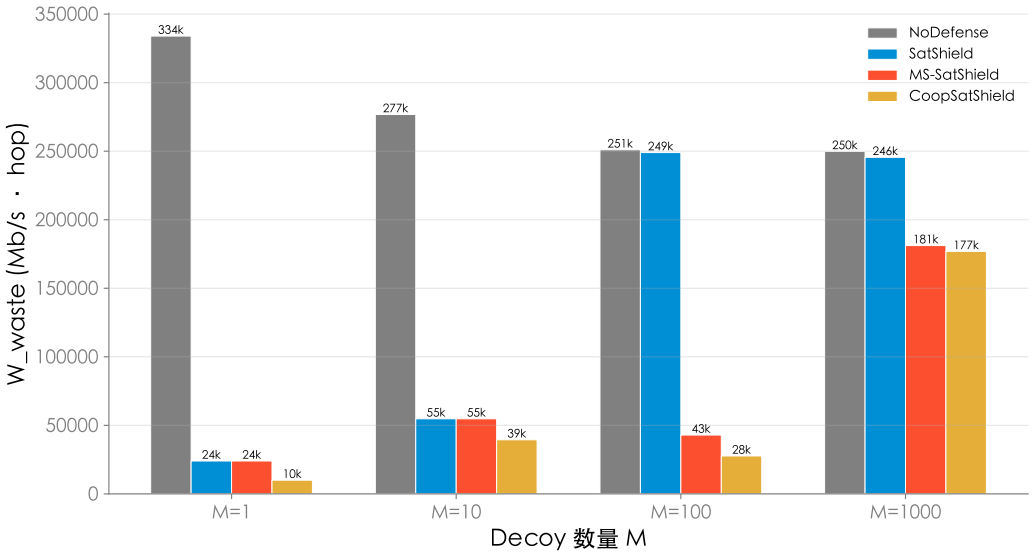


图 4-4 不同碎片化程度 M 下的无效带宽占用 W_{waste} 对比。四组分组柱状图分别对应 $M = 1, 10, 100, 1000$，每组包含 NoDefense（灰）、SatShield（蓝）、MS-SatShield（红）、CoopSatShield（金）四种方案，柱顶标注具体数值。

有效” 到” 需要多签名补偿” 再到” 候选集瓶颈” 的完整证据链，与第 第三章 章的退化分析保持一致。

（2）良性吞吐与反应时间

图 4-5给出了在 $M = 100$ 攻击场景下，良性吞吐随时间（epoch）变化的完整生命周期。实验设置攻击窗口为 epoch 30 至 epoch 120（共 90 个 epoch），图中粉色背景区域标注攻击持续阶段，并以竖虚线标注” 攻击开始”” 首次检测” 与” 推回生效” 三个关键时刻。

攻击前（epoch 0∼29），所有方案的良性吞吐均稳定在 1.0 附近（±1.5% 自然波动），符合网络正常运行预期。

攻击触发与检测延迟（epoch 30）：由于检测算法需要至少 1 个 epoch 的统计积累，攻击开始后的第一个 epoch 内所有方案均无法检出攻击流，良性吞吐统一跌至 NoDefense 水平 ≈ 0.71。这一” 检测延迟 dip” 反映了在网检测的内在时滞，对所有基线方案均成立。

首次检测生效（epoch 31）：SatShield 的 rate-only 检测在 $M = 100$ 下完全失效（F1= 0.017），吞吐维持在 NoDefense 水平（稳态均值 0.712 ± 0.016）。MS-SatShield 凭借多签名检测效果（F1= 0.909），通过高优先级队列（80

推回生效（epoch 32+）：gossip 消息完成 ROI 内扩散后，CoopSatShield 的

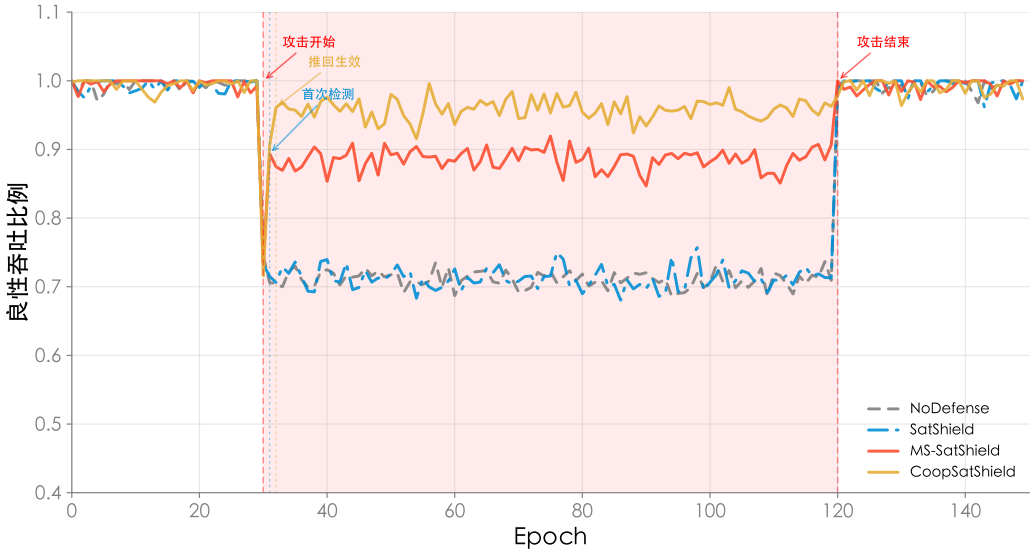


图 4-5 良性吞吐随时间变化（$M = 100$）。粉色背景标注攻击持续阶段（epoch 30∼120），竖虚线标注”攻击开始”“首次检测”“推回生效”三个关键时刻。CoopSatShield 经历两步递升：检测恢复至 0.89，推回进一步至 0.96。

pushback 规则在上游节点安装生效，约 86% 的已检测攻击流在上游被丢弃，释放了中间链路带宽。稳态下 CoopSatShield 吞吐恢复至 0.959 ± 0.015，相比 MS-SatShield 额外提升 7.3 个百分点。推回覆盖率未达 100%（约 14% 攻击流通过非推回路径到达目标链路），这使得 CoopSatShield 的稳态吞吐为 0.959 而非理想的 1.0，体现了分布式推回的工程约束。

攻击结束（epoch 120 后）：所有方案吞吐迅速恢复至 1.0 附近。总结而言，CoopSatShield 的吞吐恢复呈现清晰的*两步递升*过程：第一步由检测触发的队列调度提供基础保护（+17.5 pp），第二步由推回触发的上游丢弃提供增量收益（+7.3 pp），二者叠加使良性吞吐在攻击期间稳定维持在 0.96 水平。

（3）协同开销与”无信令风暴”验证

本章最关注的工程风险之一是大规模网络中的信令风暴。图 4-6以 1×3 子图展示了协同开销与抑制机制效果。

子图 (a) 对比了有抑制（$k = 3$）与无抑制基线下 EG 消息数随 ROI 半径 R 的变化。无抑制下 EG 消息从 $R = 1$ 的 45 条增长到 $R = 6$ 的 765 条（增长 17 倍），接近超线性增长。有抑制后同一区间增长从 45 条到 347 条（7.7 倍），在 $R = 4$ 时抑制使消息数从 369 降至 203，减少 45%。PT 消息数恒定为 44 条（仅沿推回路径单向传播，不受 R 影响）。子图中浅红色填充区域直观展示了抑制机制的收益。

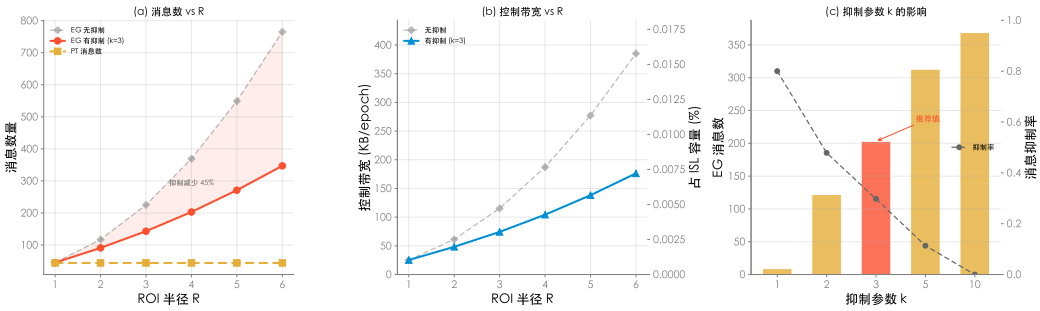


图 4-6 协同控制开销与抑制机制效果。(a)EG 消息数随 R 变化：灰色虚线为无抑制基线，红色实线为有抑制（$k=3$），填充区域为抑制收益；(b) 控制带宽随 R 变化，右轴标注占 ISL 容量百分比（$R=4$ 时仅 0.004%）；(c) 抑制参数 k 的影响，$k=3$ 为推荐值。

子图 (b) 进一步将消息数转化为控制带宽（KB/epoch），并以右侧 y 轴给出占 ISL 容量（20 Gb/s）的百分比。在推荐配置 $R=4$ 时，有抑制下控制带宽为 104 KB/epoch，仅占 ISL 容量的 0.004%；即使在 $R=6$ 的最大测试半径下，占比也不超过 0.008%。该结果定量证实了协同开销可忽略不计，验证了式(4-7)与式(4-9)的理论分析。

子图 (c) 展示了抑制参数 k 对 EG 消息数与消息抑制率的影响。$k=1$ 时抑制率高达 80%，但消息仅 9 条，覆盖不足；$k=10$ 时抑制率降至 0%，等效于无抑制。推荐 $k=3$（图中红色标注）在覆盖与开销之间取得平衡：抑制率 30%，消息数 203 条，检测效果与 $k=10$ 接近。

（4）对碎片化攻击证据的鲁棒性

图 4-7以 2×2 子图系统评估了各方案在碎片化攻击（$M \in \{1, 5, 10, 50, 100, 500, 1000\}$）下的鲁棒性。子图分别展示检测效果（F1 与 Recall）、检测精度（Precision）、良性吞吐恢复与无效带宽占用。

F1 倒 U 型趋势（子图 (a)）：MS-SatShield 与 CoopSatShield 的 F1 随 M 呈现倒 U 型，而非直觉中的单调递减。该趋势呈现三阶段行为：

（i）**精度受限段**（$M \leq 10$）：此段所有方案的 Recall 均为 1.0（虚线所示），即攻击 key 被完全捕获；然而 F1 较低（$M=1$ 时约 0.09），原因在于约 19 个良性重流 key（聚合速率 474 ∼ 606 Mb/s，超过 $\tau \cdot \text{rate}_{p99} = 242.5$ Mb/s）被恒定误判为可疑。由子图 (b) 可见，$M=1$ 时 Precision 仅 0.05（TP= 1，FP≈ 19），Precision 不足是 F1 低的唯一原因。需要强调的是，F1 低*并不代表检测失败*：$M=1$ 时唯一的攻击 key（聚合速率 10,024 Mb/s）被正确检出，子图 (c) 显示 SatShield 与 MS-SatShield 的吞吐均恢复至 0.886，CoopSatShield 通过推回进一步恢复至 1.000。

（ii）**有效检测段**（$M = 50 \sim 100$）：攻击 key 数远超 FP 数，FO/FI 特征有效区分攻击与良性流量。$M = 100$ 时 MS-SatShield 的 Precision 升至 0.833、Recall 保持 1.0，F1 达到峰值 0.909。同一区间 SatShield 完全失效（$M = 50$ 时 F1= 0.057，$M = 100$ 时 F1= 0.017），因攻击 per-key 速率已接近良性 p_{99}，rate-only 无法区分。该段结果与第 第三章 章图 3-3(b) 的“分离段”分析完全一致。

（iii）**召回受限段**（$M \geq 500$）：攻击 key 的单 key 速率降至 20 ∼ 35 Mb/s，大量脱离 top-K 候选集（TopK Recall≈ 0.30）。MS-SatShield 的 Precision 反而高达 0.936 ∼ 1.000（被筛入候选集的攻击 key 几乎全部被正确识别），但 Recall 仅 0.30，导致 F1 降至 0.445 ∼ 0.466。

CoopSatShield 与 MS-SatShield 的 F1 完全相同（子图 (a) 两条 F1 实线重合）：这是因为二者共享同一检测流水线（多签名评分器）。在当前实验的 bot 均匀分布场景下，每个节点的本地 top-K 命中率已足够高，协同证据聚合不改变单节点检测输出。需要强调的是，**本章协同机制的核心价值在于缓解层面而非检测层面**：gossip 实现跨节点的攻击信息共享，而 pushback 据此将缓解行动从受害链路前推至上游入口——这恰是引言中”仅靠本地缓解无法避免上游带宽浪费”这一问题的直接回应。CoopSatShield 的核心优势体现在*缓解层面*：子图 (c) 显示，在 $M \leq 100$ 时 CoopSatShield 的吞吐恢复至 1.000，显著高于 MS-SatShield 的 0.886（差值 11.4 pp），原因是推回释放了上游链路带宽；子图 (d) 显示 CoopSatShield 的 W_{waste} 在各 M 值下始终最低，与图 4-4的渐进退化分析一致。当 $M \geq 500$ 时所有方案吞吐均退化（CoopSatShield 降至 0.80），反映了候选集覆盖率瓶颈对缓解效果的传导。值得指出的是，在 bot 分布极度不均的特定场景下（例如攻击流量集中经过少数中继节点且这些节点的本地候选集无法完全覆盖），协同聚合有潜力提升检测 Recall；但在本章实验采用的均匀 bot 分布假设下，该效果未显现。这一局限将在第4.4节进行讨论。

4.4 本章小结

本章面向大规模低轨卫星网络的链路洪泛攻击，进一步从“单星本地闭环防御”推进到“多星协同在网计算与推回缓解”。针对 LEO 网络中控制环延迟高、拓扑快变、攻击自适应快的物理约束 [5, 10]，本章提出 CoopSatShield：以第第三章章本地多签名检测为基础，将检测输出结构化为可协同的证据单元；设计抑制型邻居扩散协议，通过事件驱动、ROI 限制与冗余抑制三重机制避免大规模网络中的信令风暴；并提出逐跳 pushback 机制，利用入端口贡献分布实现无需全局逆路由的推回缓解，将攻击抑制点推回至上游入口，从而降低网络内部无效带宽占

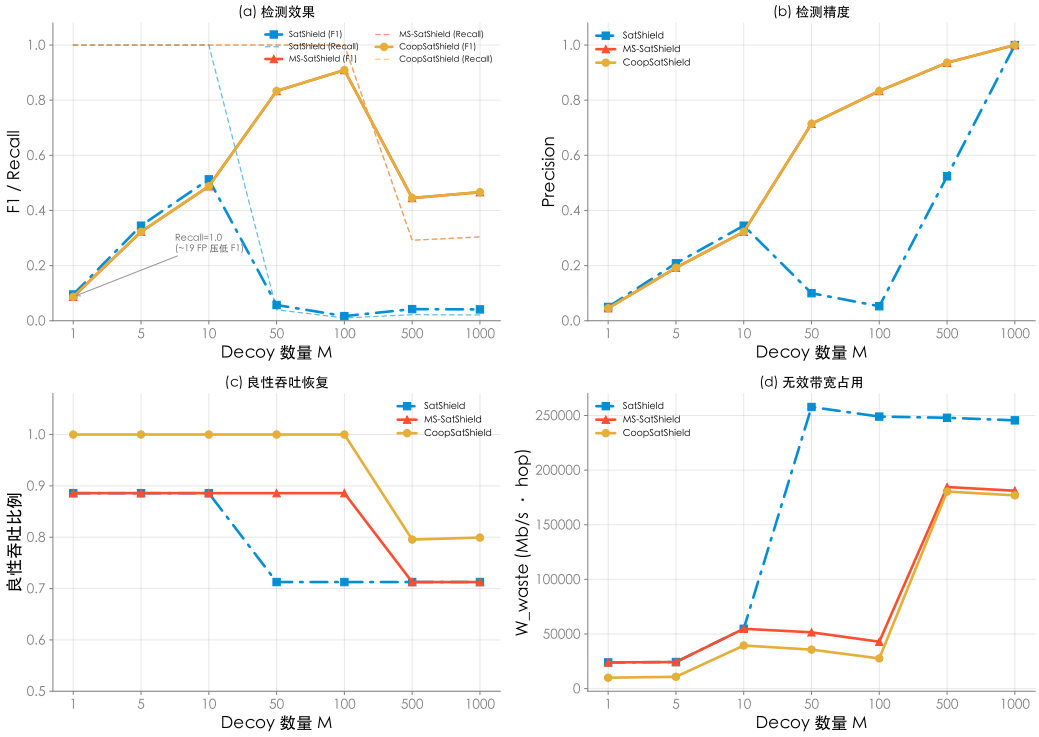


图 4-7 碎片化攻击下的鲁棒性（2×2 子图）。(a)F1（实线）与 Recall（虚线）随 M 变化，注意 $M \leq 10$ 时 F1 低是 Precision 不足所致，Recall 始终为 1.0；(b)Precision 随 M 变化；(c) 良性吞吐恢复；(d) 无效带宽 W_{waste}。

用并提升良性吞吐保护。

实验结果表明：（1）在推回缓解方面，CoopSatShield 在 $M = 100$ 时将 W_{waste} 降至 NoDefense 的 11.0%，相比 MS-SatShield（17.1%）进一步降低 6.1 个百分点；（2）在良性吞吐保护方面，攻击期间良性吞吐稳态维持在 0.959±0.015，相比 MS-SatShield（0.886）提升 7.3 个百分点，且检测延迟仅 1 epoch 即开始恢复；（3）在协同开销方面，推荐配置 $R = 4, k = 3$ 下控制带宽仅占 ISL 容量的 0.004%，抑制机制使 EG 消息数比无抑制基线减少 45%，证实无信令风暴；（4）在碎片化攻击鲁棒性方面，$M = 100$ 时 F1 达到峰值 0.909，在 $M \leq 100$ 范围内良性吞吐保持 1.000，展现了协同推回对缓解效果的显著增益。上述结果为全文“面向大规模低轨卫星的在网计算研究”提供了从*单节点在网计算*到*多节点协同在网计算*的完整技术链条。

局限性与未来方向。本章工作仍存在以下局限，可为后续研究提供出发点：（1）*协同不提升检测 F1*：在本章实验采用的均匀 bot 分布假设下，协同聚合评分 S^{net} 未改变单节点的检测输出，协同的核心贡献限于缓解层面（pushback）。在 bot 分布极度不均或攻击流量集中经过少数中继节点的场景下，协同聚合有潜力提升检测 Recall，但该场景有待验证。（2）*候选集覆盖率瓶颈*：$M \geq 500$ 时大量攻击 key 的单 key 速率低于 top-K 门限而脱离候选集，导致所有方案的 F1 与吞吐保护均退化。

解决该瓶颈需要在“候选集规模”与“数据面资源消耗”之间寻找更优的折中方案。（3）*安全性假设*：本章假设所有邻居节点均可信，未考虑恶意节点注入虚假 EG/PT 消息的场景。针对此类拜占庭攻击，未来可引入基于轻量级签名或声誉机制的消息验证策略。此外，跨轨道面协同与 SDN 控制面混合架构的对接也是值得进一步探索的方向。

参考文献

[1] Kodheli O, Lagunas E, Maturo N, et al. Satellite Communications in the New Space Era: A Survey and Future Challenges[J]. IEEE Communications Surveys & Tutorials, 2021, 23(1): 70–109.

[2] Wang S, Li Q. Satellite computing: Vision and challenges[J]. IEEE Internet of Things Journal, 2023, 10(24): 22514–22529.

[3] Chaudhry A U, Yanikomeroglu H. Laser intersatellite links in a starlink constellation: A classification and analysis[J]. IEEE Vehicular Technology Magazine, 2021, 16(2): 48–56.

[4] Bhattacherjee D, Singla A. Network topology design at 27,000 km/hour[C]. Proceedings of the 15th International Conference on Emerging Networking Experiments And Technologies, New York, NY, USA: ACM, 2019: 341–354.

[5] Giuliari G, Ciussani T, Perrig A, et al. ICARUS: Attacking low Earth orbit satellite networks[J]. 2021.

[6] Kang M S, Lee S B, Gligor V D. Crossfire: An analysis of Firefox extension-reuse vulnerabilities[C]. IEEE Symposium on Security and Privacy (S&P), 2013: 127–141. Crossfire: attack against network infrastructure via link flooding.

[7] Studer A, Perrig A. Coremelt: An attack on the internet core through legitimate traffic[C]. European Symposium on Research in Computer Security (ESORICS), Springer, 2009: 209–225.

[8] Lu T, Ding X, Shang J, et al. DoSat: A DDoS Attack on the Vulnerable Time-Varying Topology of LEO Satellite Networks. Applied Cryptography and Network Security, Cham: Springer Nature Switzerland, vol. 14584, 2024, 265–282. http://dx.doi.org/10.1007/978-3-031-54773-7_11.

[9] Jiang W. Software defined satellite networks: A survey[J]. Digital Communications and Networks, 2023, 9(6): 1243–1264.

[10] Jiang W, Jiang H, Xie Y, et al. SatShield: In-Network Mitigation of Link Flooding Attacks for LEO Constellation Networks[J]. IEEE Internet of Things Journal, 2024.

[11] Liu Z, Namkung H, Nikolaidis G, et al. Jaqen: A high-performance switch-native approach for detecting and mitigating volumetric DDoS attacks with programmable switches[C]. 30th USENIX Security Symposium, 2021: 3829–3846.

[12] Kianpisheh S, Taleb T. A Survey on In-Network Computing: Programmable Data Plane and Technology Specific Applications[J]. IEEE Communications Surveys & Tutorials, 2023, 25(1): 701–761.

[13] Sapio A, Abdelaziz I, Aldilaijan A, et al. In-Network Computation is a Dumb Idea Whose Time Has Come[C]. Proceedings of the 16th ACM Workshop on Hot Topics in Networks, New York, NY, USA: ACM, 2017: 150–156.

[14] Hauser F, Häberle M, Merling D, et al. A survey on data plane programming with P4: Fundamentals, advances, and applied research[J]. Journal of Network and Computer Applications, 2023, 212: 103561.

[15] Yue P, An J, Zhang J, et al. Low earth orbit satellite security and reliability: Issues, solutions, and the road ahead[J]. IEEE Communications Surveys & Tutorials, 2023, 25(3): 1604–1652.

[16] Guo W, Xu J, Pei Y, et al. A distributed collaborative entrance defense framework against DDoS attacks on satellite internet[J]. IEEE Internet of Things Journal, 2022, 9(17): 15497–15510.

[17] Xue L, Ma X, Luo X, et al. LinkScope: Toward detecting target link flooding attacks[J]. IEEE Transactions on Information Forensics and Security, 2018, 13(10): 2423–2438.

[18] Wang L, Li Q, Jiang Y, et al. Woodpecker: Detecting and mitigating link-flooding attacks via SDN[J]. Computer Networks, 2018, 147: 1–13.

[19] Zheng J, Li Q, Gu G, et al. Realtime DDoS defense using COTS SDN switches via adaptive correlation analysis[J]. IEEE Transactions on Information Forensics and Security, 2018, 13(7): 1838–1853.

[20] Xing J, Wu W, Chen A. Ripple: A programmable, decentralized link-flooding defense against adaptive adversaries[C]. 30th USENIX Security Symposium, 2021: 3865–3881.

[21] Zhou H, Hong S, Liu Y, et al. Mew: Enabling large-scale and dynamic link-flooding defenses on programmable switches[C]. 2023 IEEE Symposium on Security and Privacy (SP), 2023: 3178–3192.

[22] Ilha A D S, Lapolli A C, Marques J A, et al. Euclid: A Fully In-Network, P4-Based Approach for Real-Time DDoS Attack Detection and Mitigation[J]. IEEE Transactions on Network and Service Management, 2021, 18(3): 3121–3139.

[23] Alcoz A G, Strohmeier M, Lenders V, et al. Aggregate-based congestion control for pulse-wave DDoS defense[C]. Proceedings of the ACM SIGCOMM 2022 Conference, New York, NY, USA: ACM, 2022: 693–706.

[24] Wu J, Pan H, Cui P, et al. Patronum: In-network volumetric DDoS detection and mitigation with programmable switches[C]. Computer Security – ESORICS 2024, Cham: Springer Nature Switzerland, 2024: 187–207.

[25] Walker J G. Satellite constellations[J]. Journal of the British Interplanetary Society, 1984, 37: 559–571.

[26] Kassing S, Bhattacherjee D, Águas A B, et al. Exploring the “internet from space” with hypatia[C]. Proceedings of the ACM Internet Measurement Conference, Virtual Event USA: ACM, 2020: 214–229.

[27] Bosshart P, Gibb G, Kim H S, et al. Forwarding metamorphosis: Fast programmable match-action processing in hardware for SDN[C]. Proceedings of the ACM SIGCOMM 2013 Conference, ACM, 2013: 99–110.

[28] McKeown N, Anderson T, Balakrishnan H, et al. OpenFlow: Enabling innovation in campus networks[J]. ACM SIGCOMM Computer Communication Review, 2008, 38(2): 69–74.

[29] Bosshart P, Daly D, Gibb G, et al. P4: Programming protocol-independent packet processors[J]. ACM SIGCOMM Computer Communication Review, 2014, 44(3): 87–95.

[30] Bloom B H. Space/time trade-offs in hash coding with allowable errors[J]. Communications of the ACM, 1970, 13(7): 422–426.

[31] Cormode G, Muthukrishnan S. An improved data stream summary: The count-min sketch and its applications[J]. Journal of Algorithms, 2005, 55(1): 58–75.

[32] Metwally A, Agrawal D, El Abbadi A. Efficient computation of frequent and top-k elements in data streams[C]. International Conference on Database Theory (ICDT), Springer, 2005: 398–412.

[33] Whang K Y, Vander-Zanden B T, Taylor H M. A linear-time probabilistic counting algorithm for database applications[J]. ACM Transactions on Database Systems, 1990, 15(2): 208–229.

[34] Demers A, Greene D, Hauser C, et al. Epidemic algorithms for replicated database maintenance[C]. Proceedings of the Sixth Annual ACM Symposium on Principles of Distributed Computing (PODC), ACM, 1987: 1–12.

[35] Levis P, Patel N, Culler D, et al. Trickle: A self-regulating algorithm for code propagation and maintenance in wireless sensor networks[C]. USENIX NSDI, 2004: 15–28.

附录 A 建模证明